# CENG 114 BİLGİSAYAR BİLİMLERİ İÇİN AYRIK YAPILAR

Prof. Dr. Tufan TURACI
tturaci@pau.edu.tr

- Pamukkale Üniversitesi
- Mühendislik Fakültesi
- Bilgisayar Mühendisliği Bölümü
- Hafta 15

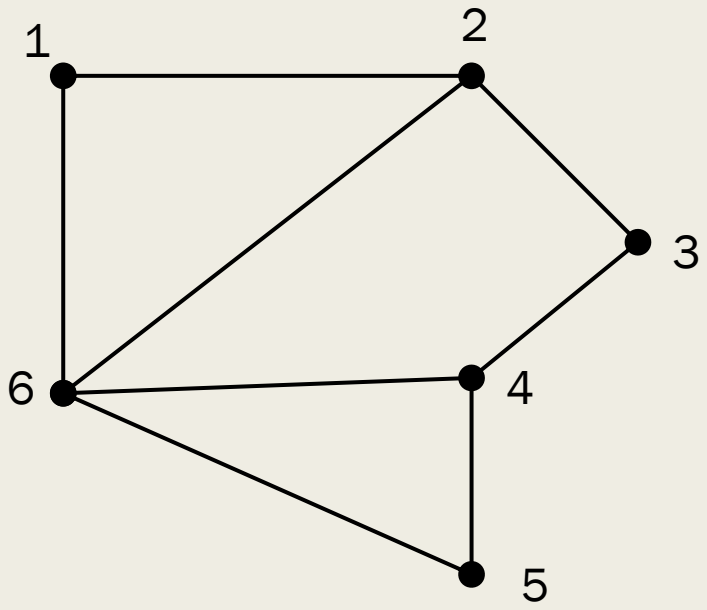
# Ders İçeriği

- **Graflarda Bağlantılılık (Graph Connectivity)**
- **Ağaçlar**

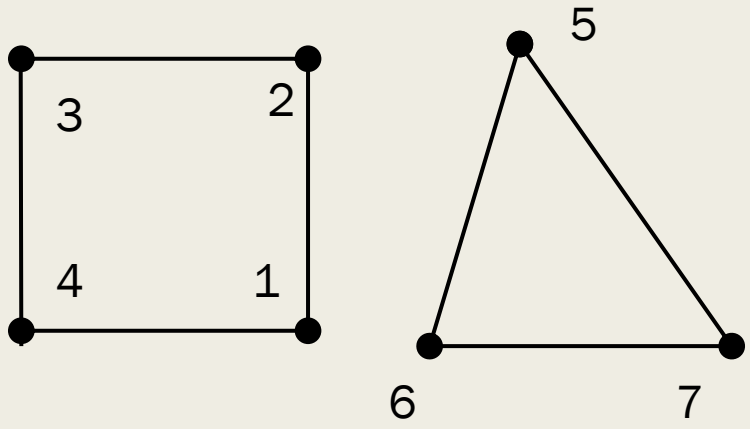
  - Ağaçların Bilgisayarlarda Saklanması

# Bağlantılılık (Connectivity)

Birleştirilmiş Graf: Bir G grafında herhangi iki tepe arasında en az bir tane yol varsa, grafın herhangi bir tepesinden diğer tüm tepelere gidilebiliyorsa bu grafa birleştirilmiş(bağlantılı) graf denir.




Birleştirilmiş Graf




Birleştirilmemiş Graf

Birleştirilmemiş bir graftaki her bir parçaya grafın bileşeni adı verilir (component).

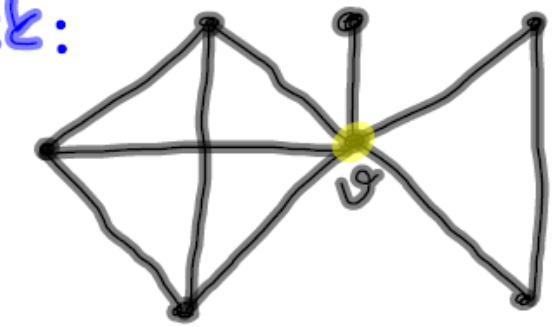
**Tanım:** Eğer G grafında (u,v)-yolu mevcut ise u,v ayrık tepeleri G grafında bağlıdır. Eğer tüm tepe çiftleri bağlıysa G grafı da bağlıdır.

**Tanım:** Eğer H grafı, bağlantılı ve G nin daha fazla tepe ya da ayrıt içeren bağantılı bir alt grafı tarafından içerilmeyen bir graf ise G grafının H alt grafına G nin bir bileşeni denir ve G nin bileşenlerinin sayısı w(G) ile gösterilir.

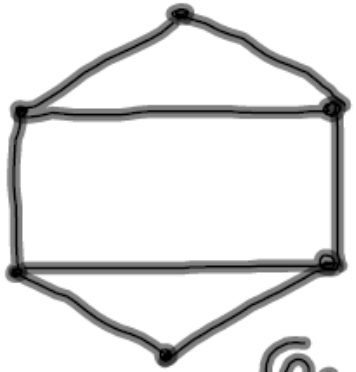
**Tanım:** Bir G grafı için $V^* \subset V(G)$ ve $E^* \subset E(G)$ olsun. Eğer $w(G-V^*) > w(G)$ ise $V^*$ bir '*vertex cut*' olarak adlandırılır. Eğer $V^*$ yalnızca bir tepeden oluşuyorsa, v bir '*cut vertex*' olarak adlandırılır. Benzer şekilde, eğer $w(G-E^*) > w(G)$ ise $E^*$ bir '*edge cut*' olarak adlandırılır. Eğer $E^*$ yalnızca bir ayrıttan oluşuyorsa, e '*cut edge*' olarak adlandırılır.

**Tanım:** Bir $G$ grafının bir $v$ tepesi için $w(G-v) > w(G)$ oluyor ise $v$'ye kesim tepe denir.

**Örnek:**




$G_1$



$G_2$

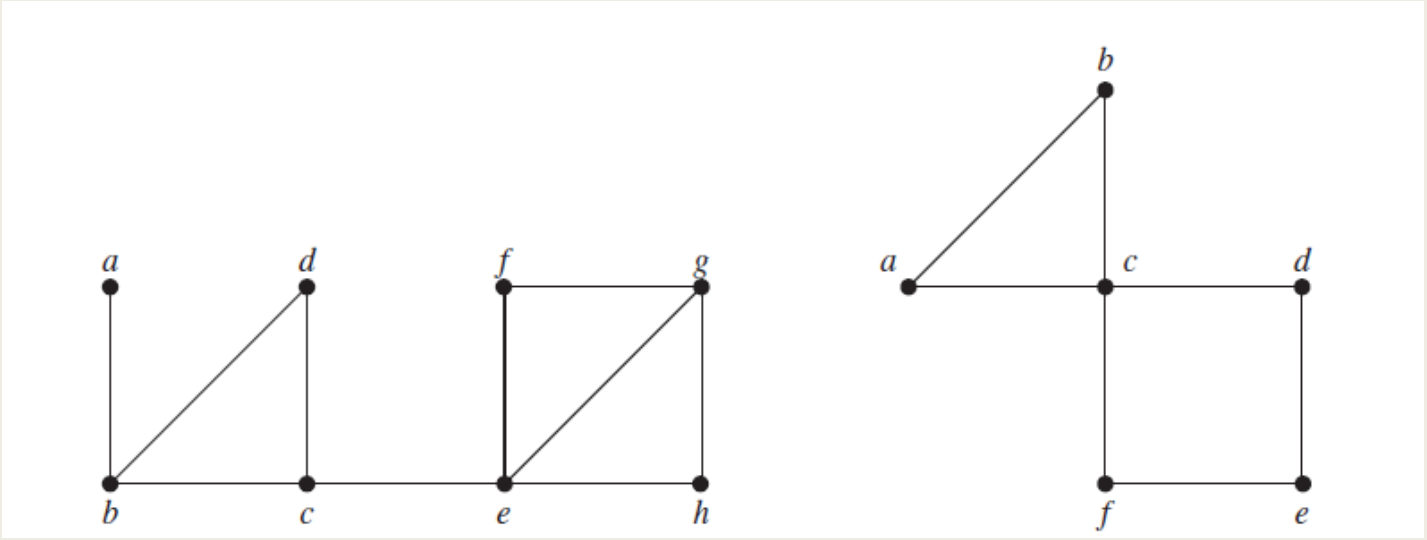
$w(G_1) = 1$

$w(G_1 - v) = 3$

$G_1$ grafında $v$ bir kesim tepedir.

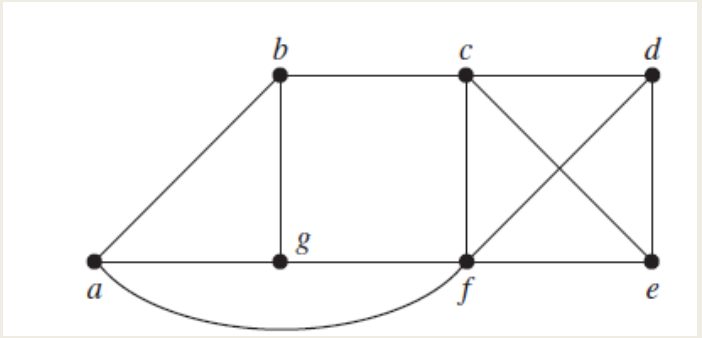
$G_2$ grafında bir kesim tepe yoktur.

--- Aşağıdaki grafları bağlantısız yapmak için graftan atılması gerekli tepeler hangileridir?

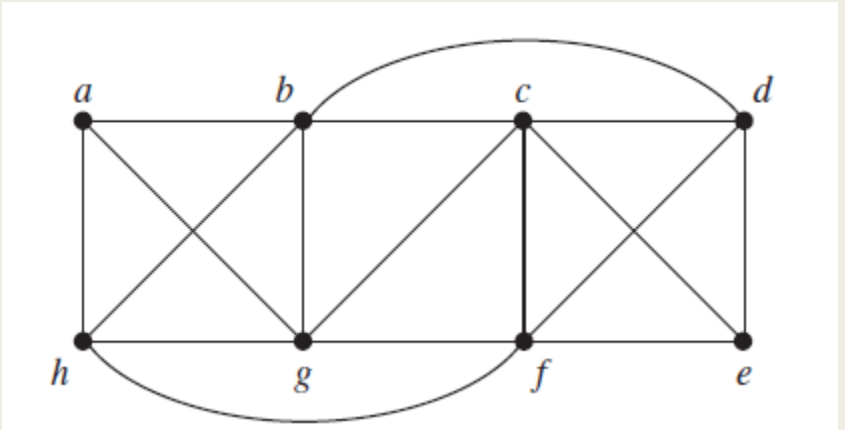



1

2

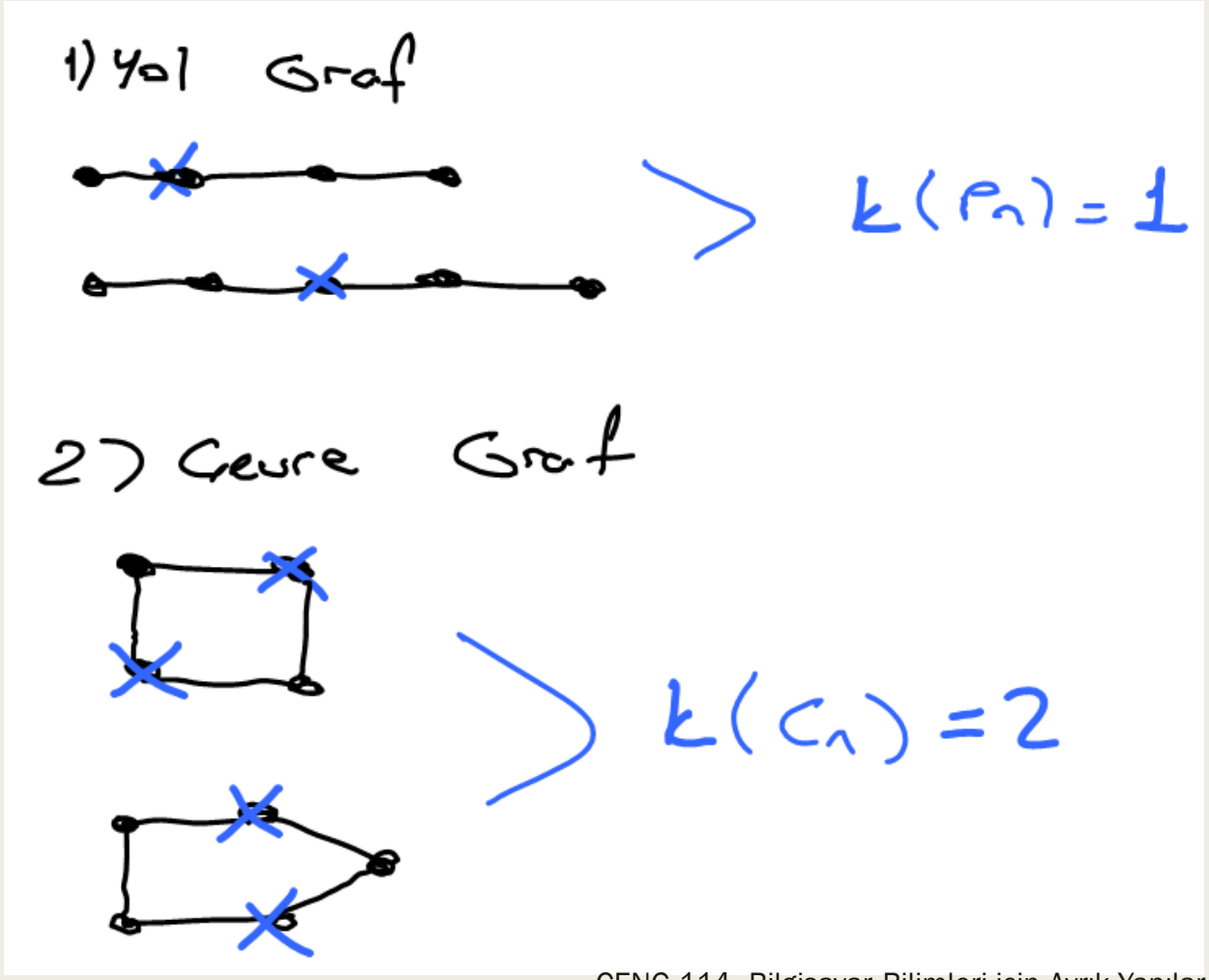



3

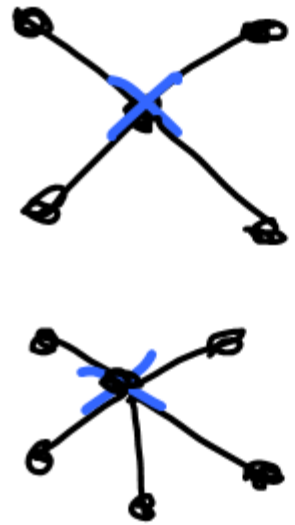



4

**Tanım:** Bağlantılı bir *G* grafını bağlantısız yapmak veya tek izole tepe elde etmek için graftan çıkarılması gereken en az tepe sayısına grafın **bağlantılılığı (connectivity)** denir ve *k(G)* ile gösterilir. Önemli grafların connectivity değerleri aşağıdadır.



3) Yıldız Graf



$k(K_{1,n}) = 1$

# 4) Tam Graf



$K_4$

sil

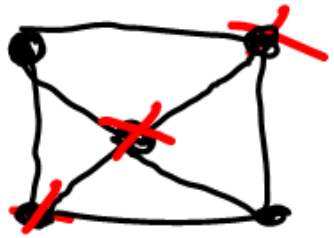


$K_3$

$k(K_N) = n-1$

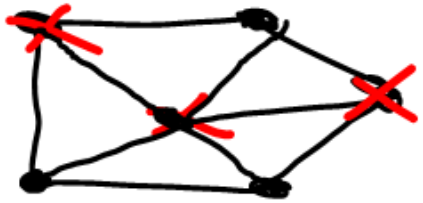
izole tepe kalmaya kadar tepe atılır.

## 5) Tekerlek Graf



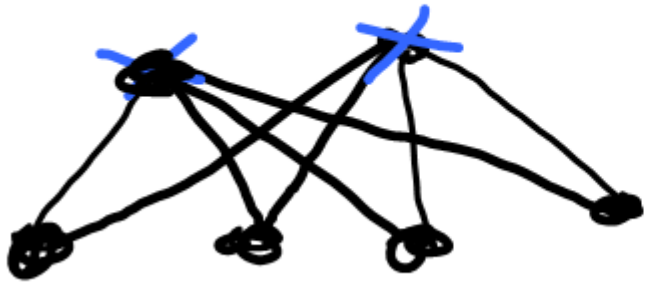
$W_{1,4}$

$k(W_{1,n}) = 3$



$W_{1,5}$

## 6) iki parçalı tam graf



$\Rightarrow k(K_{m,n}) = \min\{n,m\}$

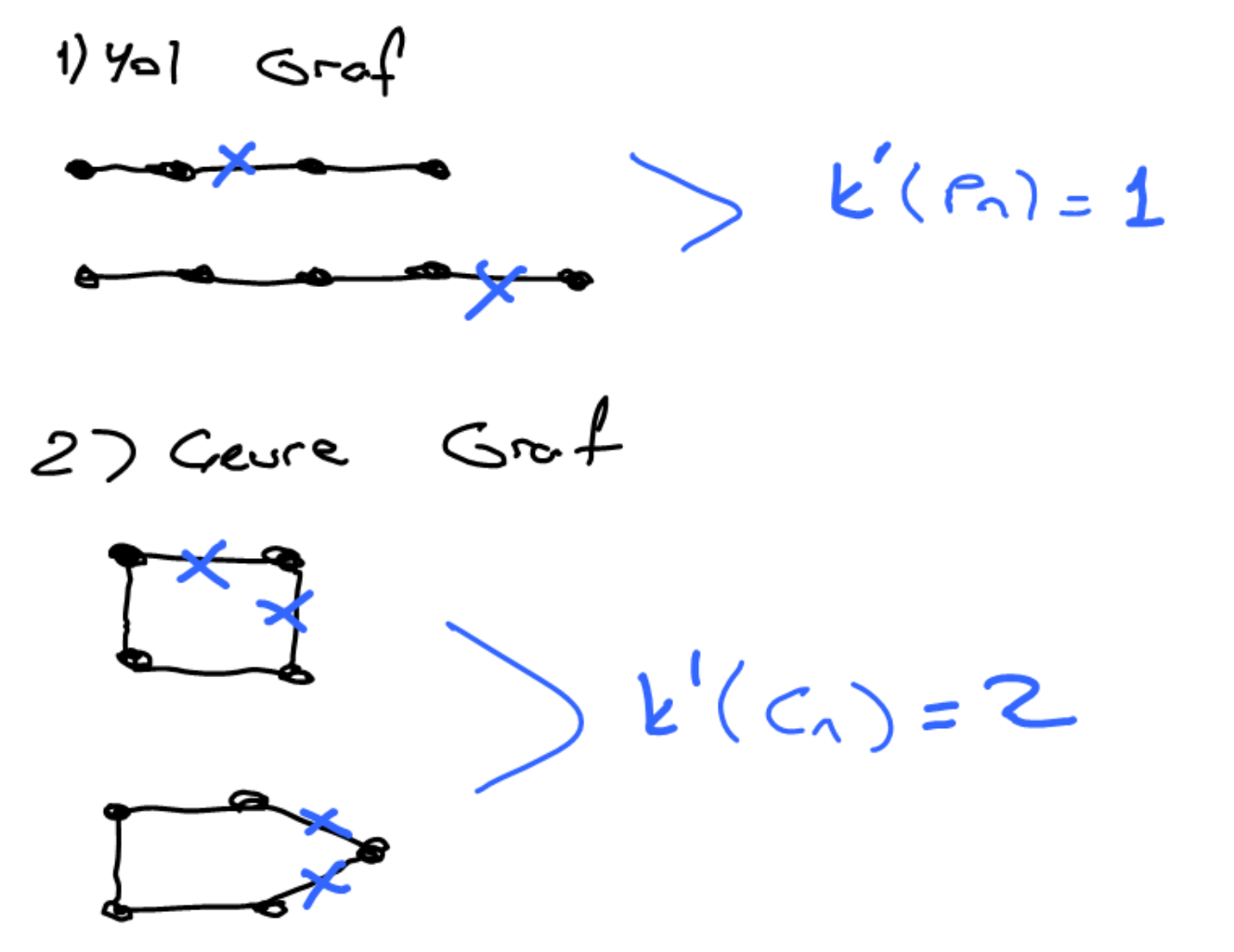
**Tanım:** Bağlantılı bir *G* grafında eğer *k*(G)=*t* ise, bu grafa *t*- bağlı graftır denir.

**Örnek 1:** *n*-tepeli tam graf, (*n*-1)- bağlı graftır.

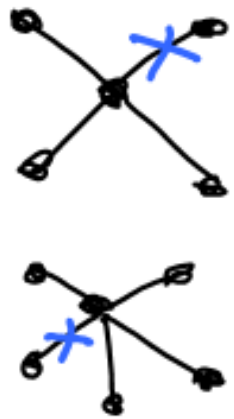
**Örnek 2:** *n*-tepeli çevre graf, *2*- bağlı graftır.

**Tanım:** Bir $G$ grafını bağlantısız yapmak için graftan çıkarılması gereken en az ayrıt sayısına ayrıt bağlantılılık sayısı (edge connectivity number) denir ve $k'(G)$ ile gösterilir.

Önemli grafların ayrıt connectivity değerleri:

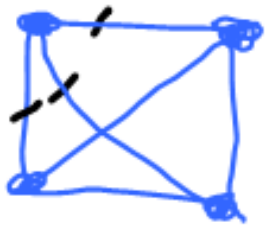


3) Yıldız Graf



$\Rightarrow$ $k'(K_{1,n}) = 1$

4) Tam Graf



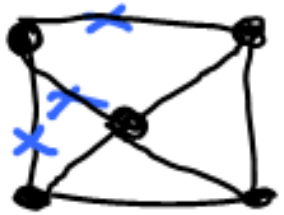
$K_4$

$\Rightarrow$ $k'(K_n) = n-1$



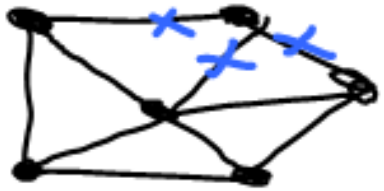
$K_3$

3) Tekerlek Graf



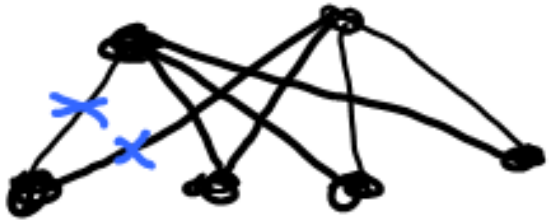
$W_{1,4}$

$k'(W_{1,n}) = 3$



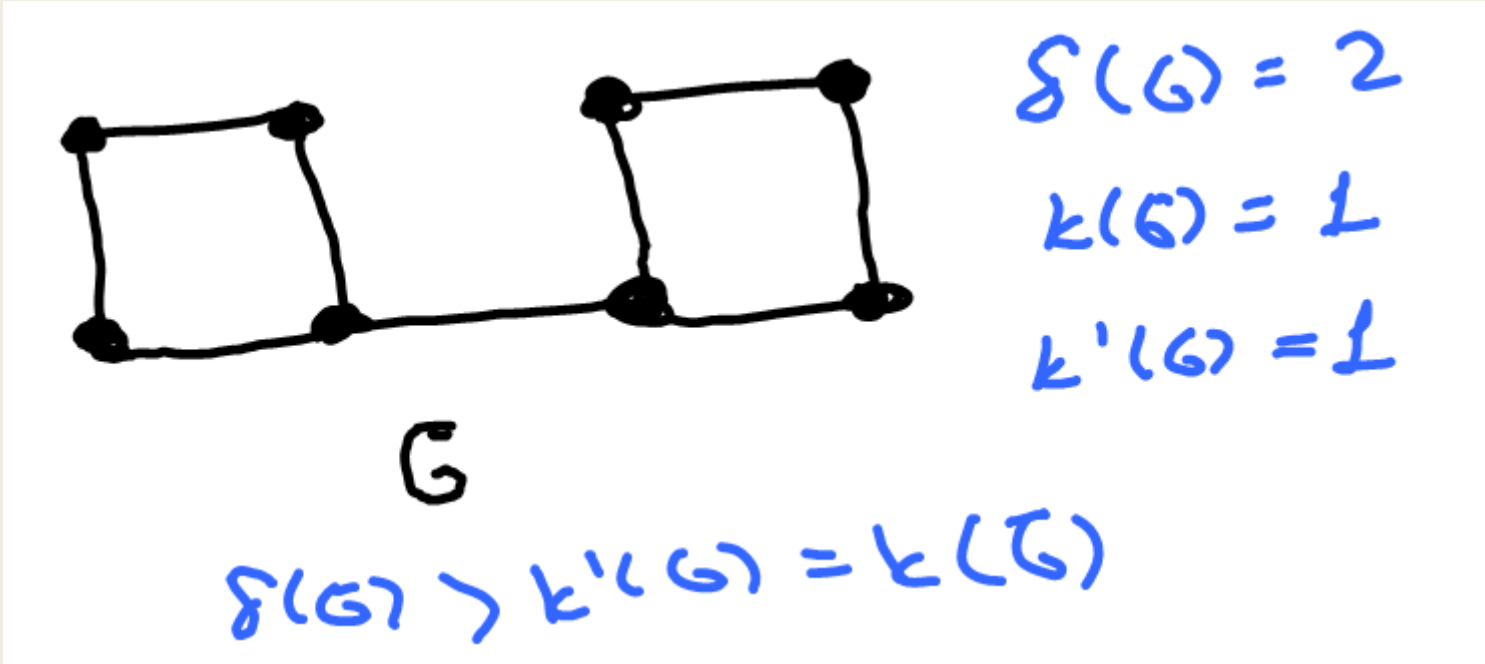
$W_{1,5}$

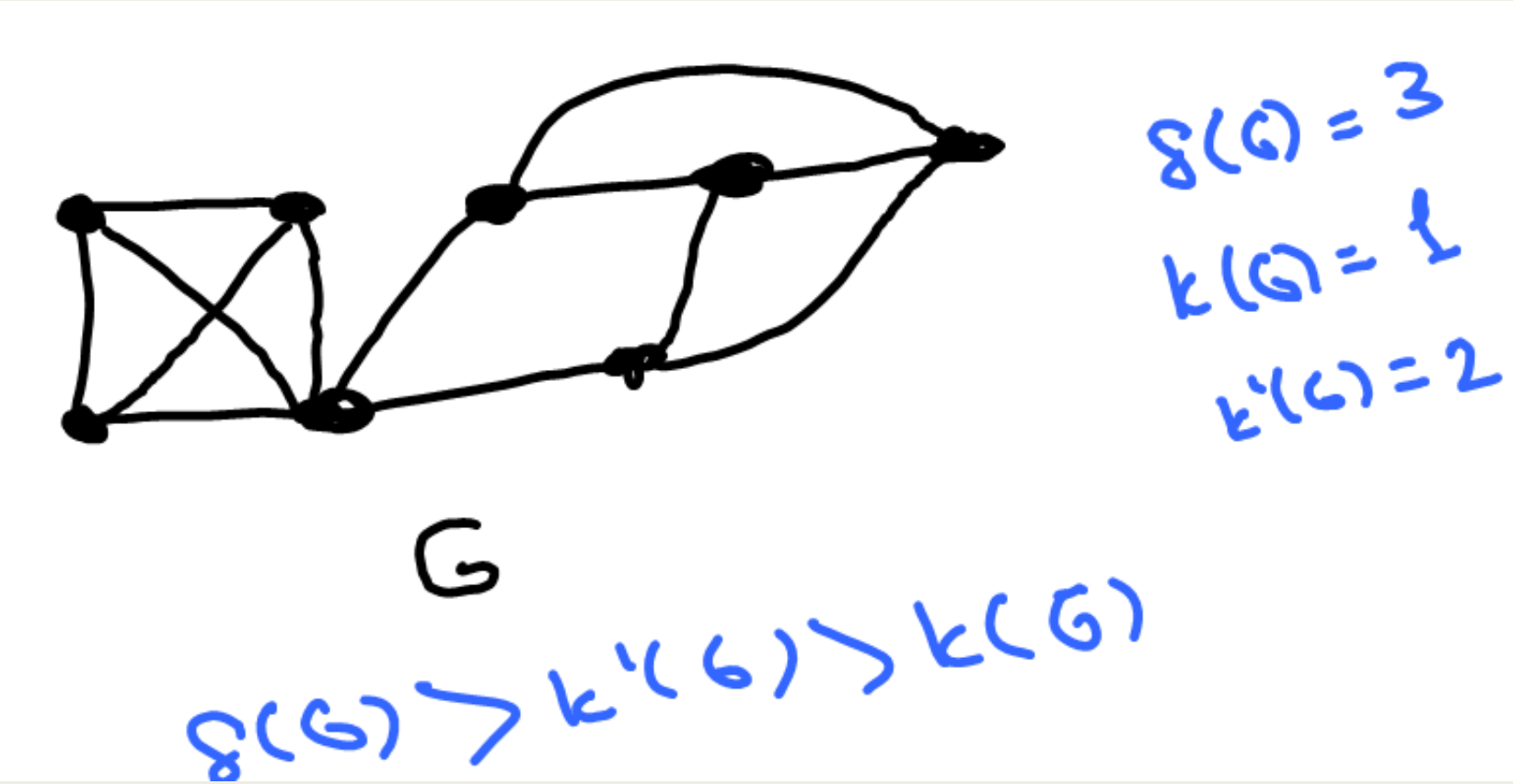
6) iki parçalı tam graf



$\Rightarrow$ $(K_{m,n}) =$ miktarım?

**Teorem :** $n$ tepeli herhangi bir $G$ grafı için, $\delta(G) \geq k'(G) \geq k(G)$ dır.



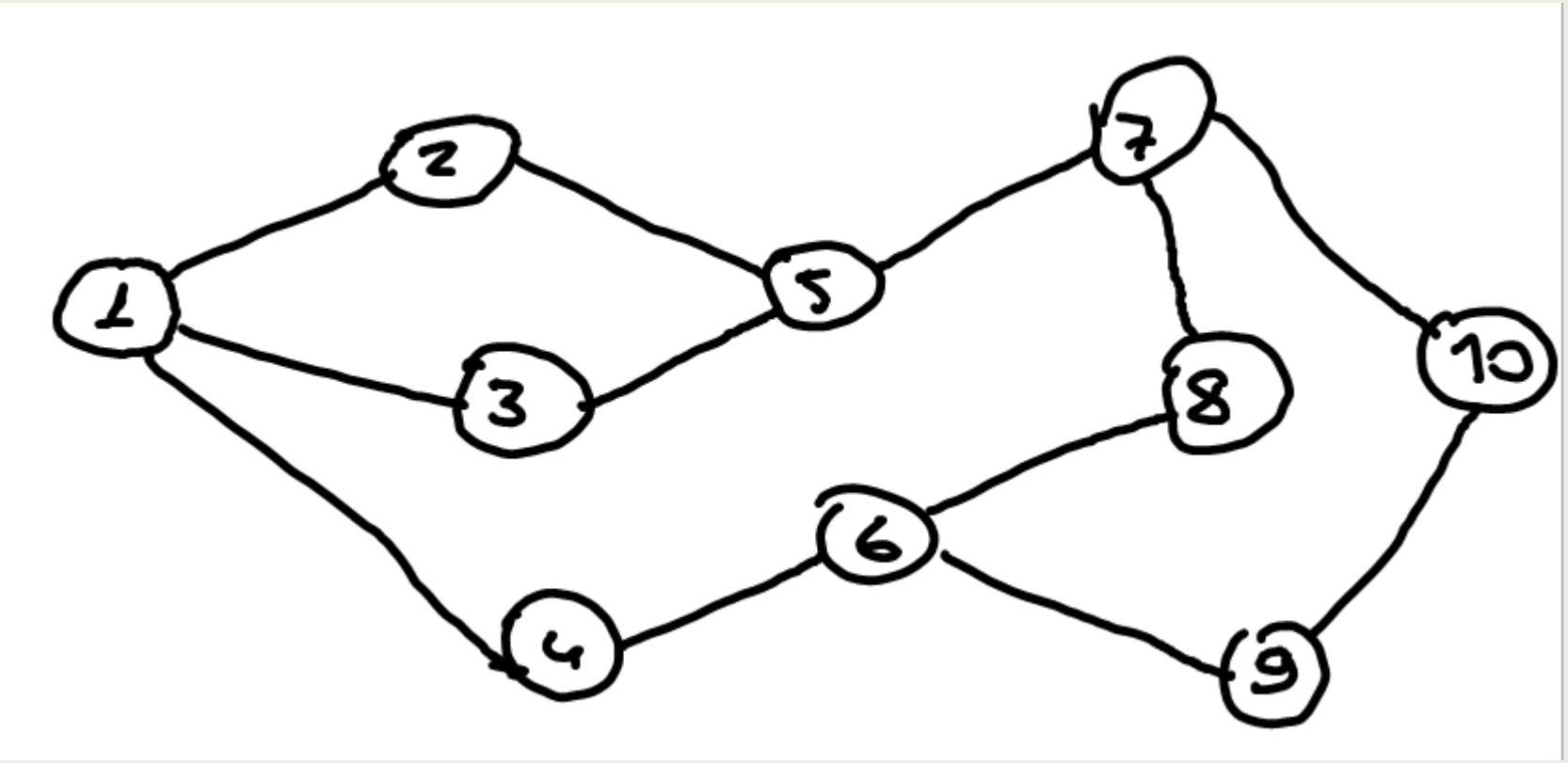

δ(G) = 3
k(G) = 1
k'(G) = 2
G
δ(G) ≥ k'(G) ≥ k(G)

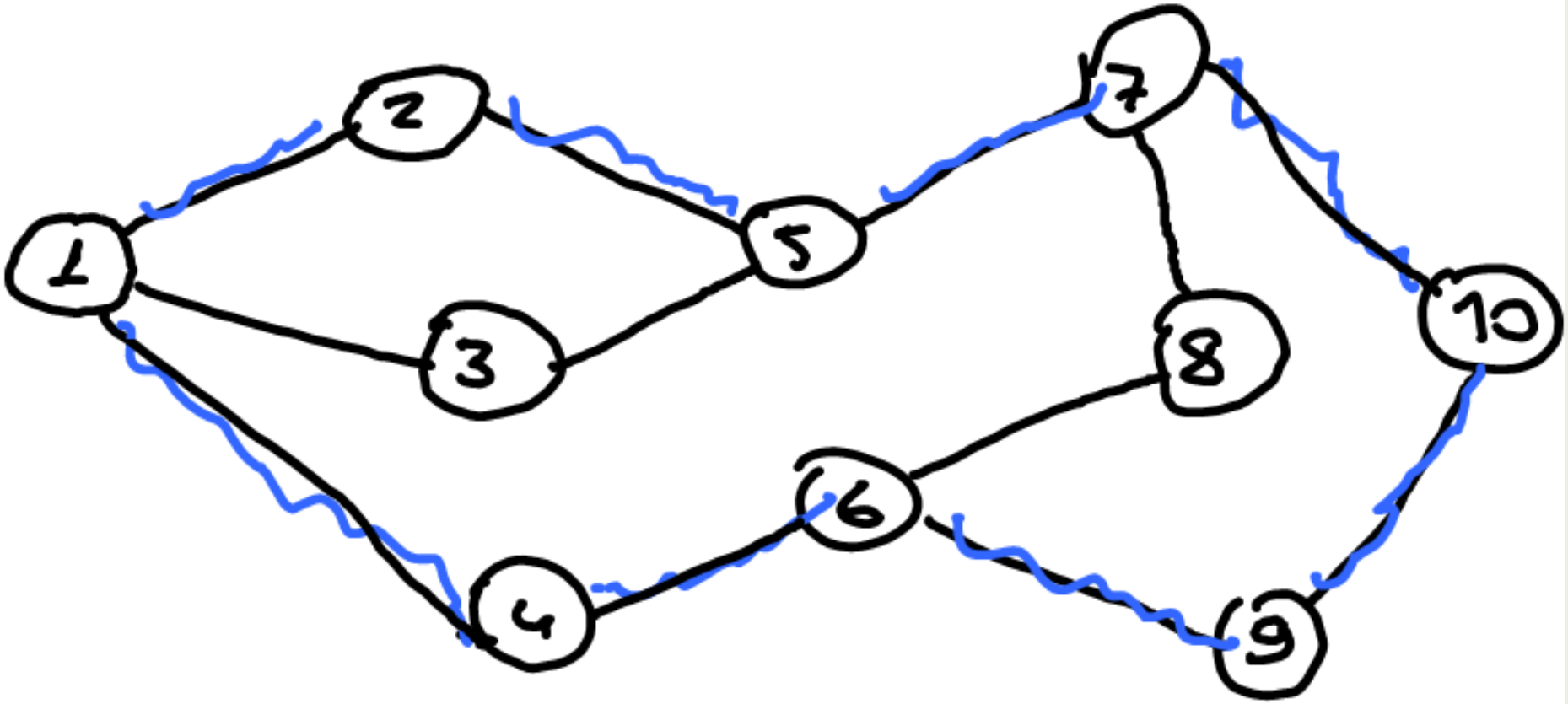
# Bir Connectivity Değerinin Bulunması

Ağlardaki maksimum akış minimum kesme ye dayanan Menger'in teoremi birleştirilmişlik konusundaki en temel teoremdir ve bu teoreme göre bir grafta bitişik olmayan *u* ve *v* tepeleri arasındaki tepe tekrarsız yolların maksimum sayısı *u* ve *v* yi bağlantısız yapmak için çizgeden atılması gereken minimum tepe sayısına eşittir. Bu haliyle teorem grafın bağlantısız olmasını sağlayan tepelerin hangileri olduğu ile ilgili bir bilgi vermemektedir.

**Teorem .** *Bağlantılı bir G grafında ayrık ve komşu olmayan iki tepe u ve v olsun. Buna göre, G'deki içten ayrık u — v yollarının maksimum sayısı, u'yu v'den ayırmak için atılması gerekli tepelerin minimum sayısına eşittir.*

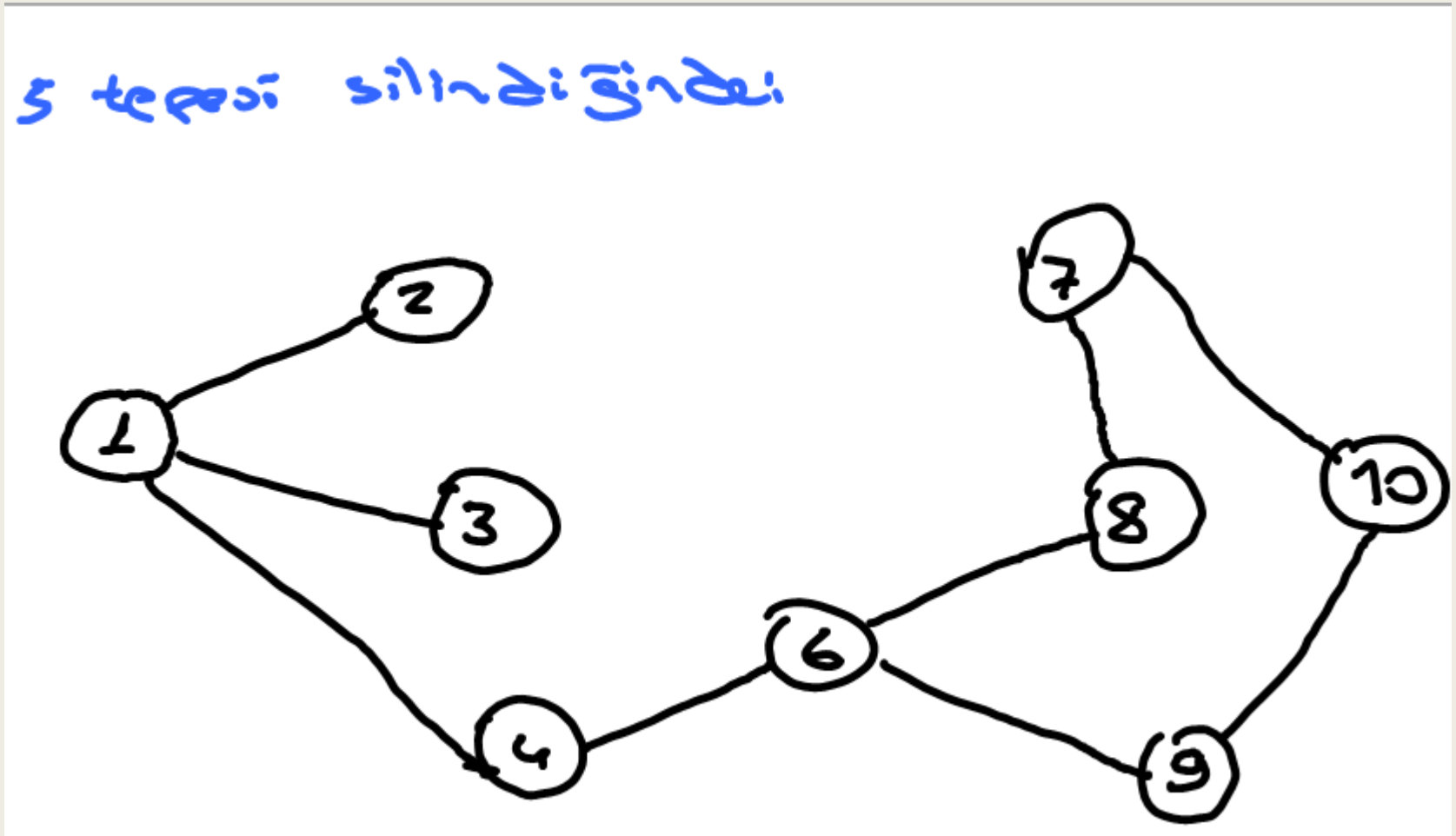
ÖRNEK: Aşağıda 10 tepeli bir G grafı verilmiştir. İçten ayrık yolların sayısı nedir?



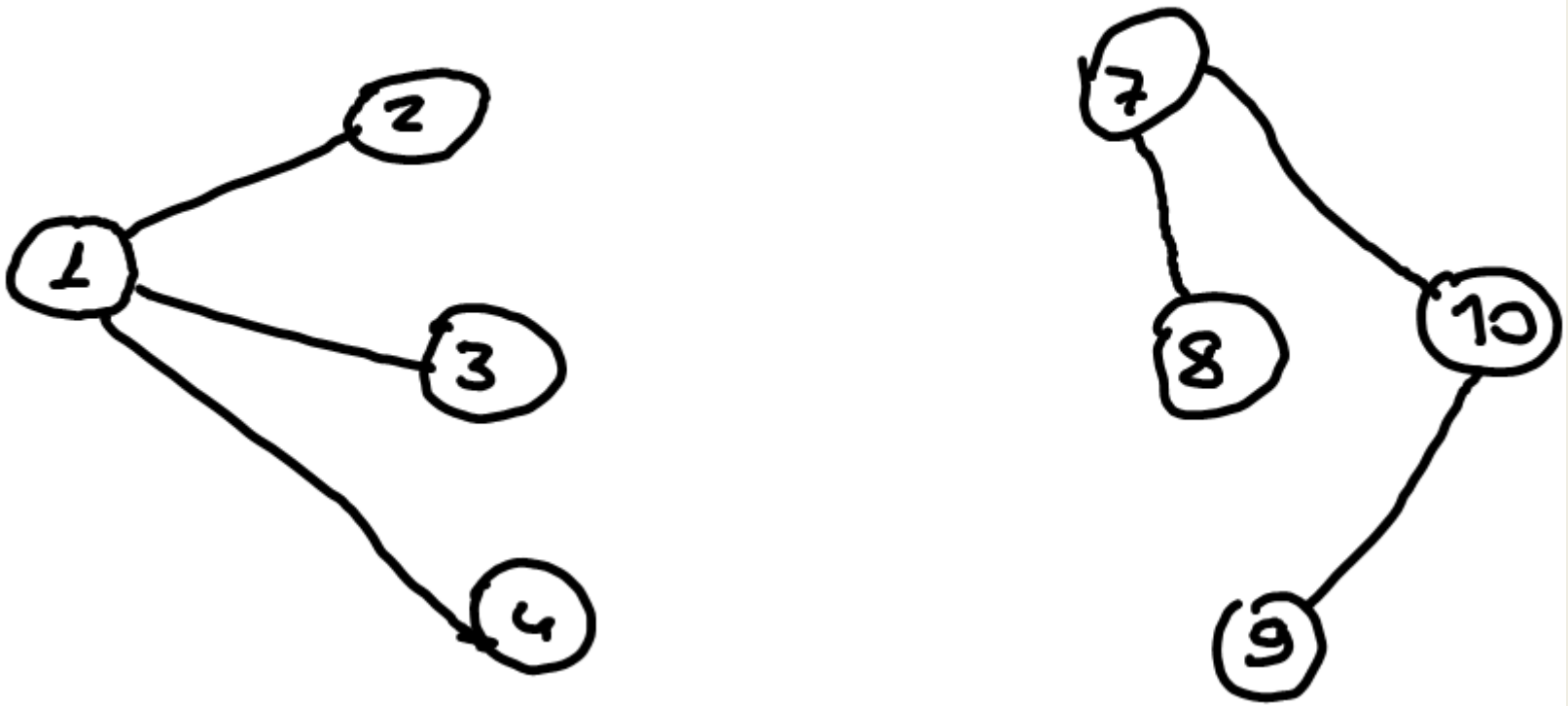



1,2,5,7,10 } yollar 5 ve 6 tepelerini kullanan
1,4,6,9,10 } işten geçmiş yollardır.

5 ve 6 tepelerini kullanan farklı
iki ayrık yollarda bulunabilir.
Fakat bunların sayısı maksimum
2'dir.
Böylece $k(G) = 2$ elde edilir.


5 tepesi silindiğinde
1
2
3
4
6
7
8
9
10

5 ve 6 tepeleri silindiğinde:




~~A~~ içten-dışa yolların sayısı

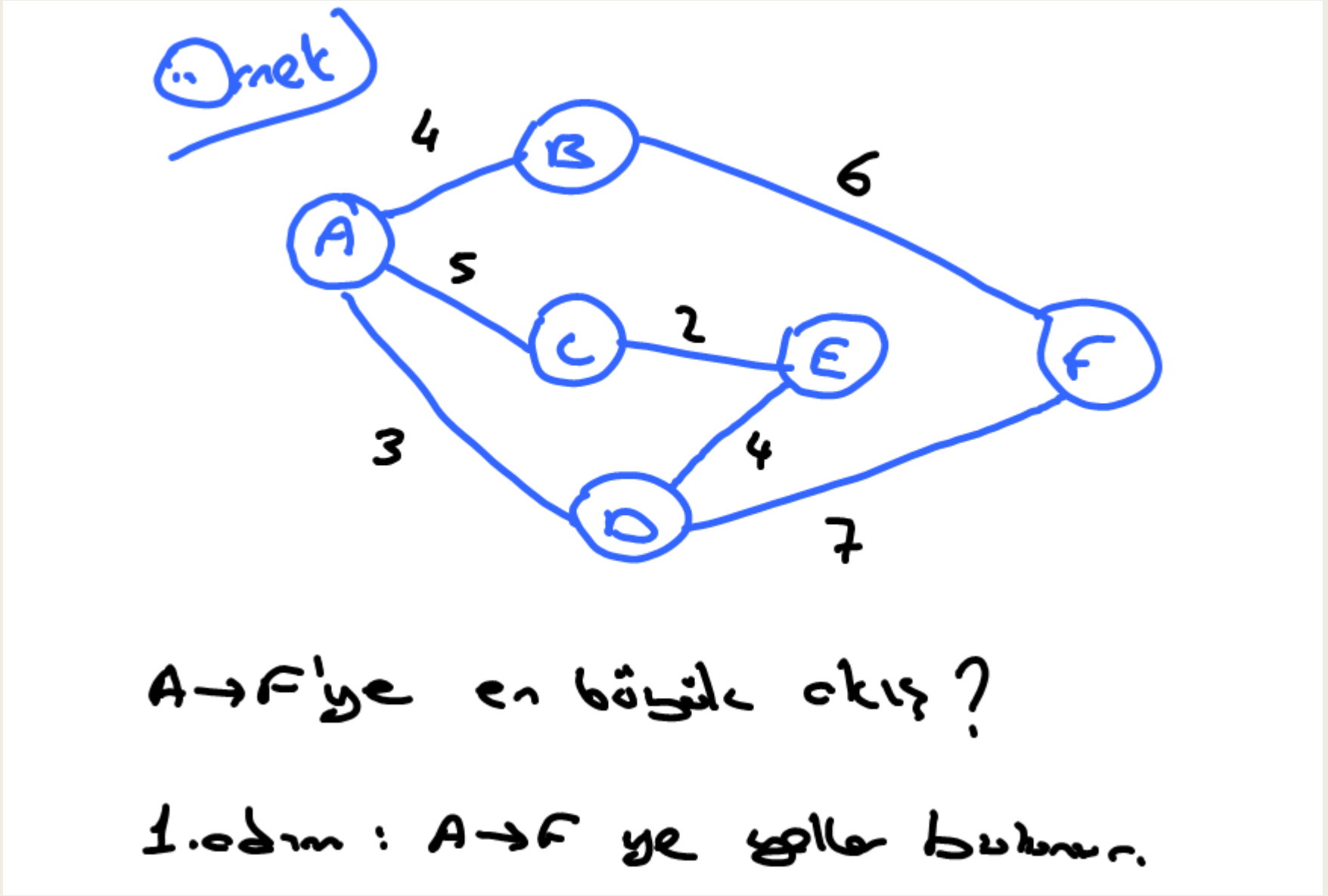
Ford-Fulkerson (en büyük akış)

algoritması yardımıyla bulunabilir.

## Ford-Fulkerson Sözde Kodu

```
Ford-Fulkerson(G, s, t)
for each edge (u,v) ∈ E(G)
    f[u,v] = 0
    f[v,u] = 0
end
```

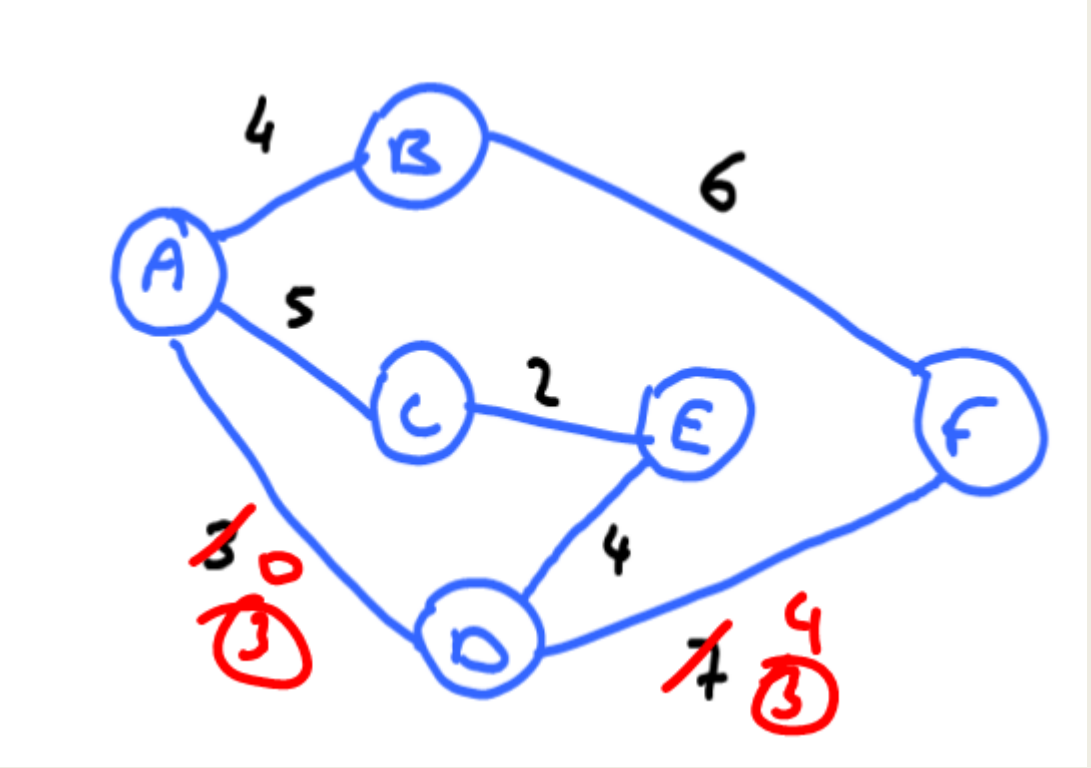
```
while (s'den t'ye p yolu var ise)
    cf(p) = min{ cf(u,v) | (u,v) ∈ p }
    for each edge (u,v) in p
        f(u,v) = f(u,v) + cf(p)
        f(v,u) = - f(u,v)
    end
end
```


Örnek
A
B
C
D
E
F
4
6
5
2
3
4
7
A→F'ye en büyük akış?
1.adım : A→F ye yollar bulunur.

ADF yolu

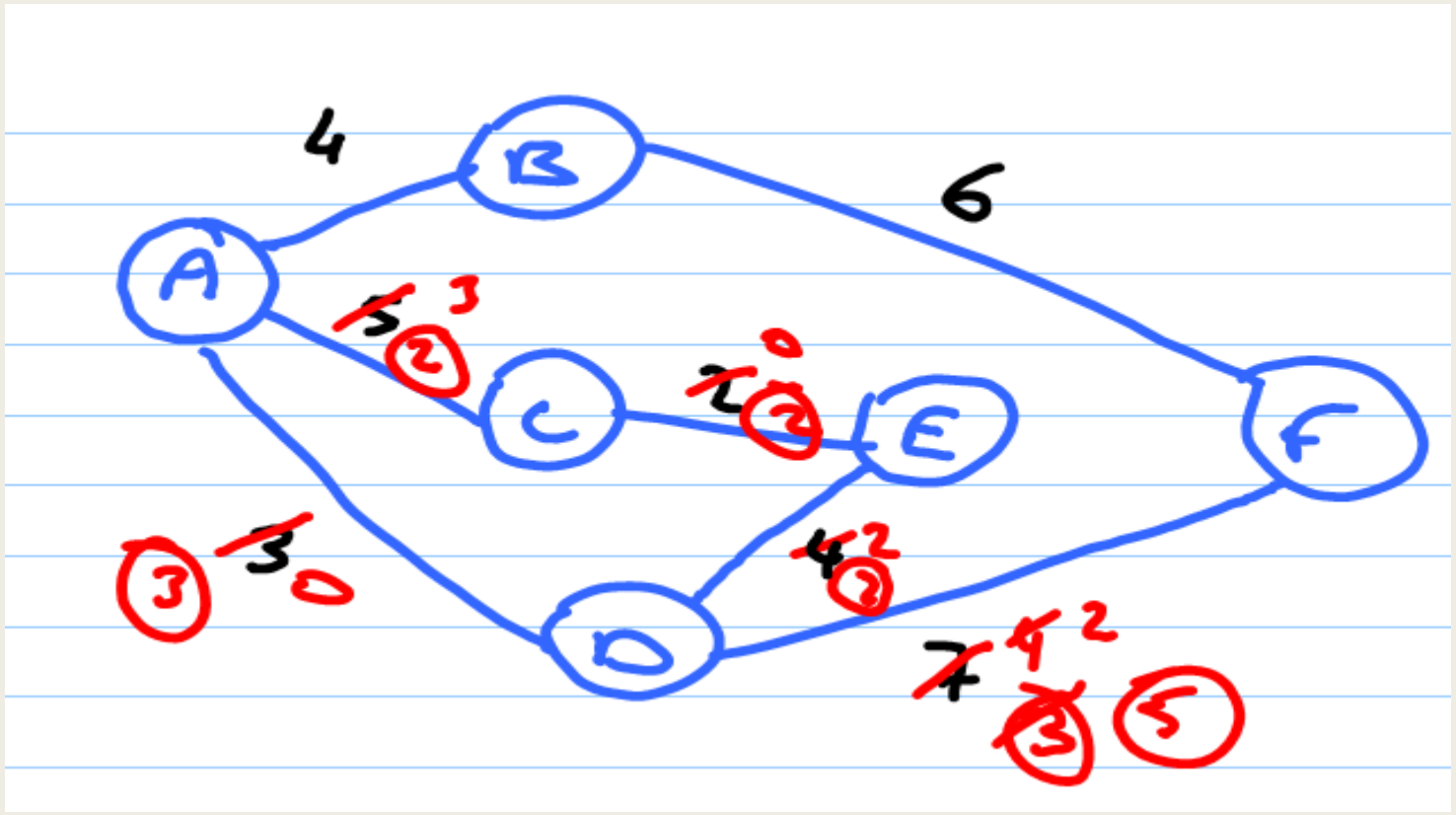
$\min\{A\text{-}D, D\text{-}F\} = 3$ //

3 7



2. adım: A C E D f yolu

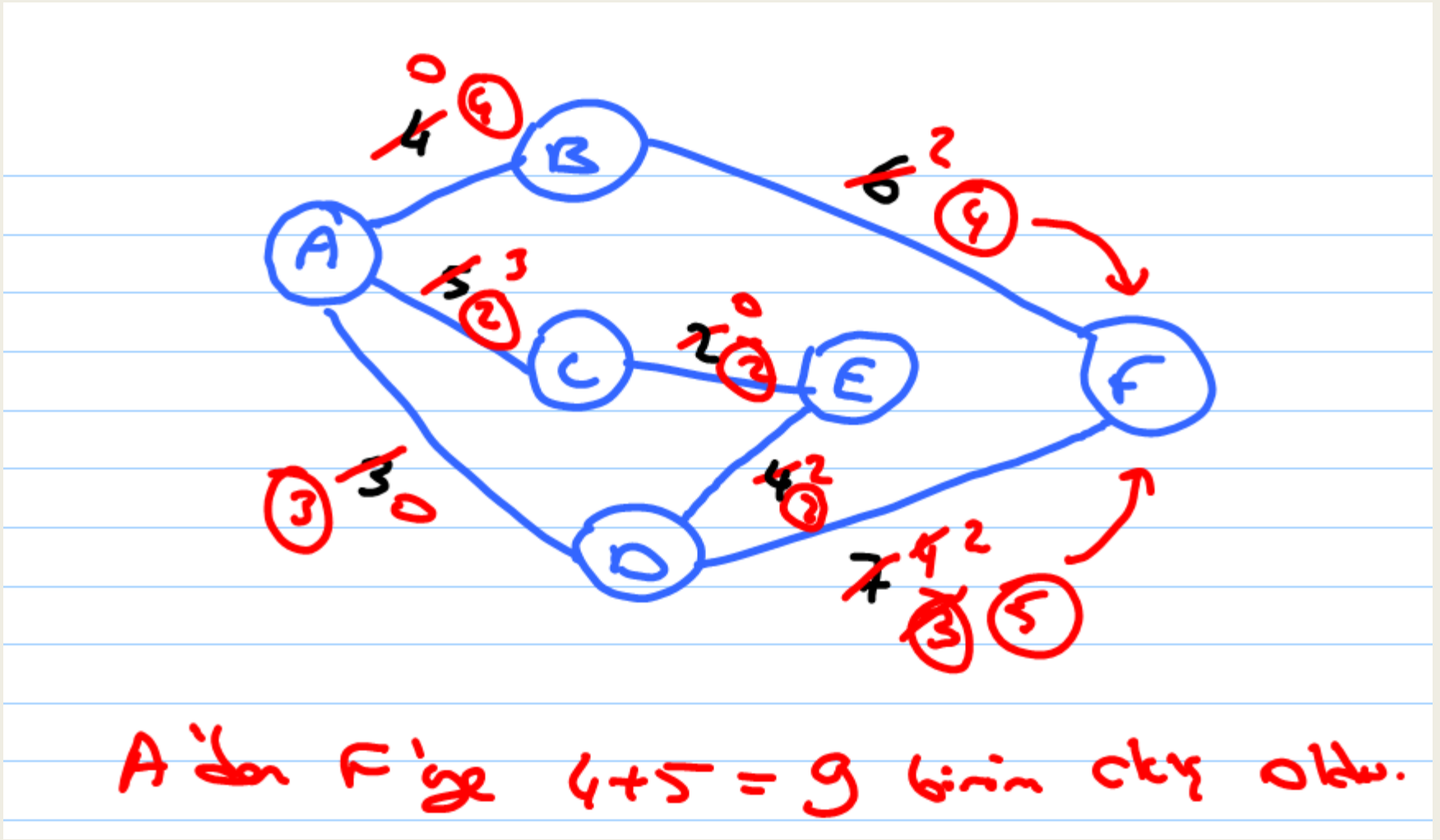
$\min\{A\text{-}C, C\text{-}E, E\text{-}D, D\text{-}f\} = \min\{5, 2, 4, 4\} = 2$



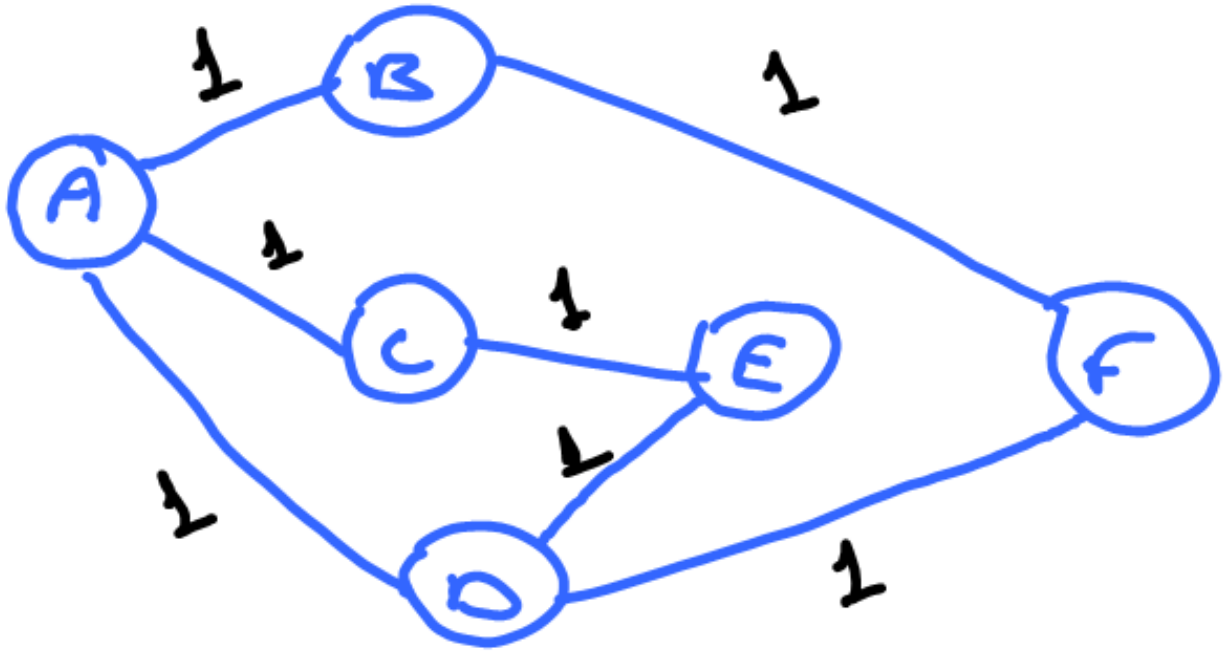
3. adım

ABF yolu

$\min\{A\text{–}B, B\text{–}F\} = \min\{4, 6\} = 4$

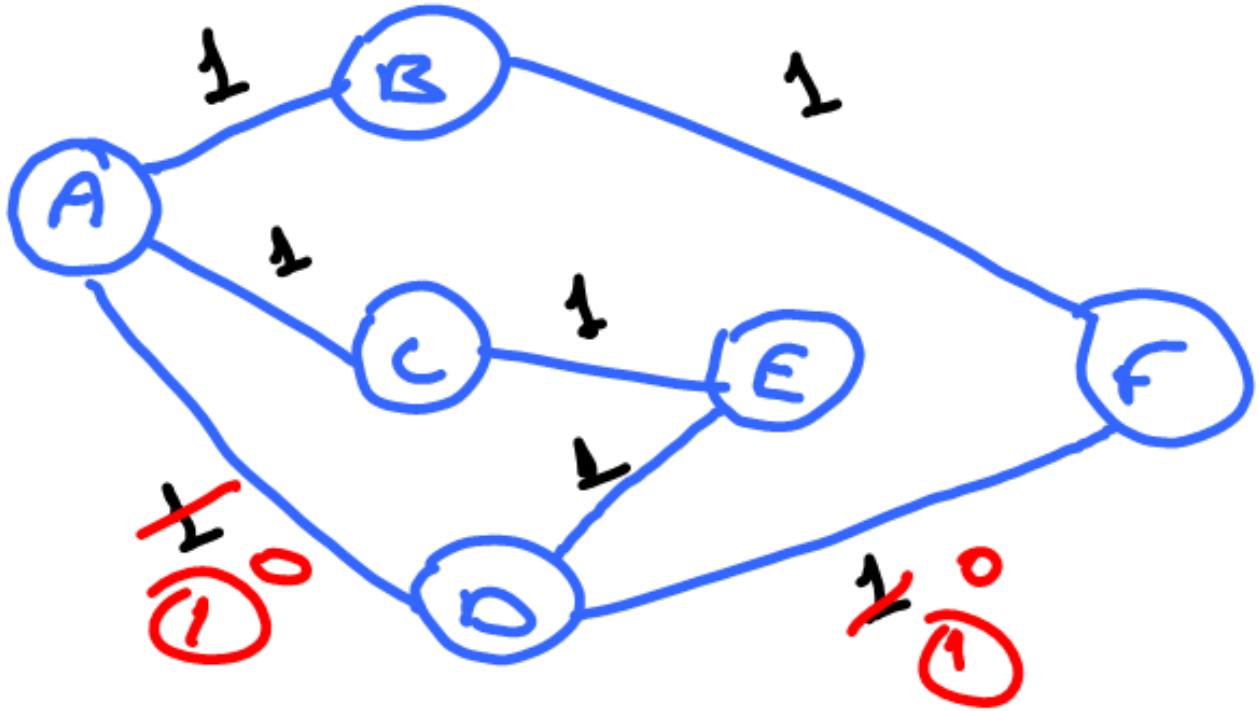


* Tüm akış değerlerine 1 verdiğimizde, aşağıdaki graf elde edilir.



1. adım: ADF yolu

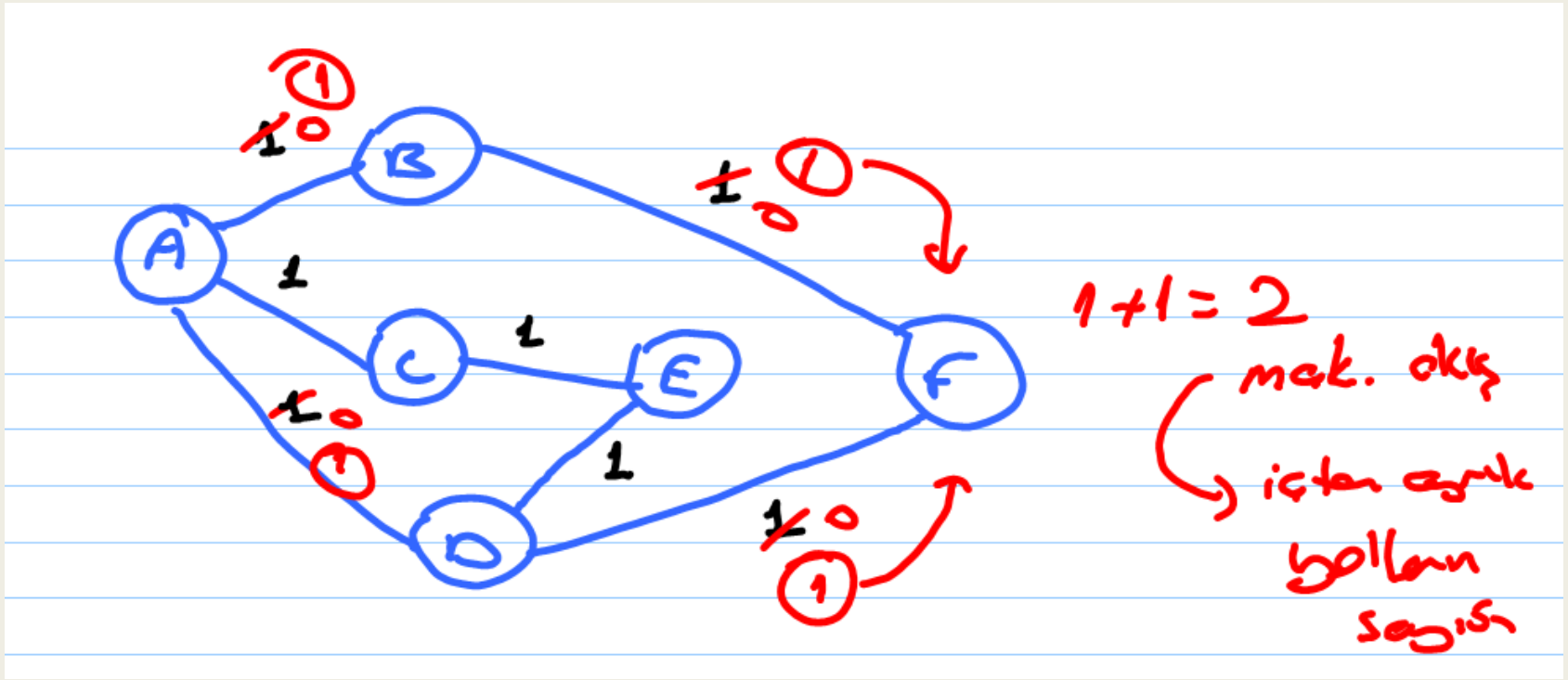
$\min\{A\text{-}D, D\text{-}F\} = \min\{1, 1\} = 1$



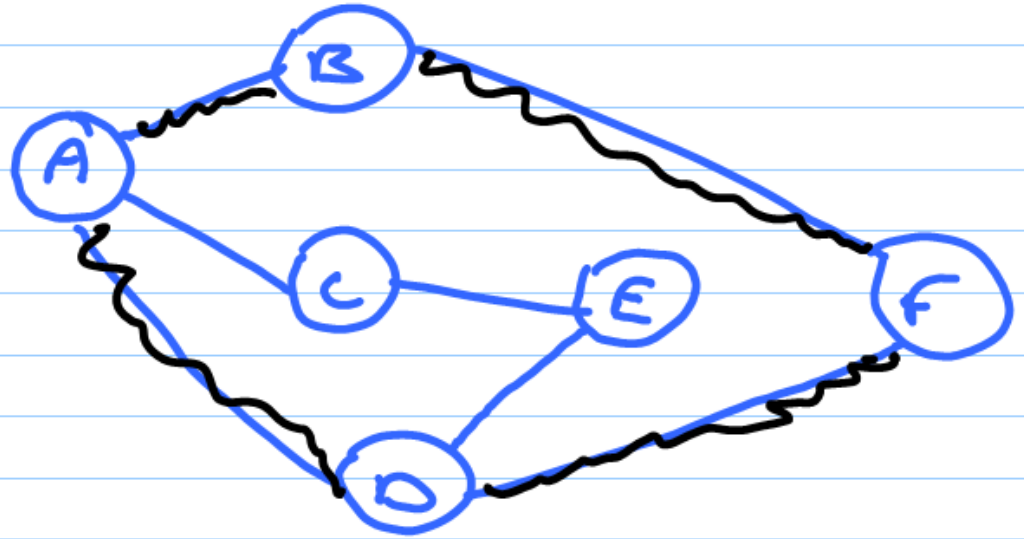
2. adım: ACEDF yolu

$\min\{A\text{-}C, C\text{-}F, E\text{-}D, D\text{-}F\} = \min\{1, 1, 1, 0\} = 0$

3. adım: ABF yolu $\min\{A\text{-}B, B\text{-}F\} = \min\{1, 1\} = 1$



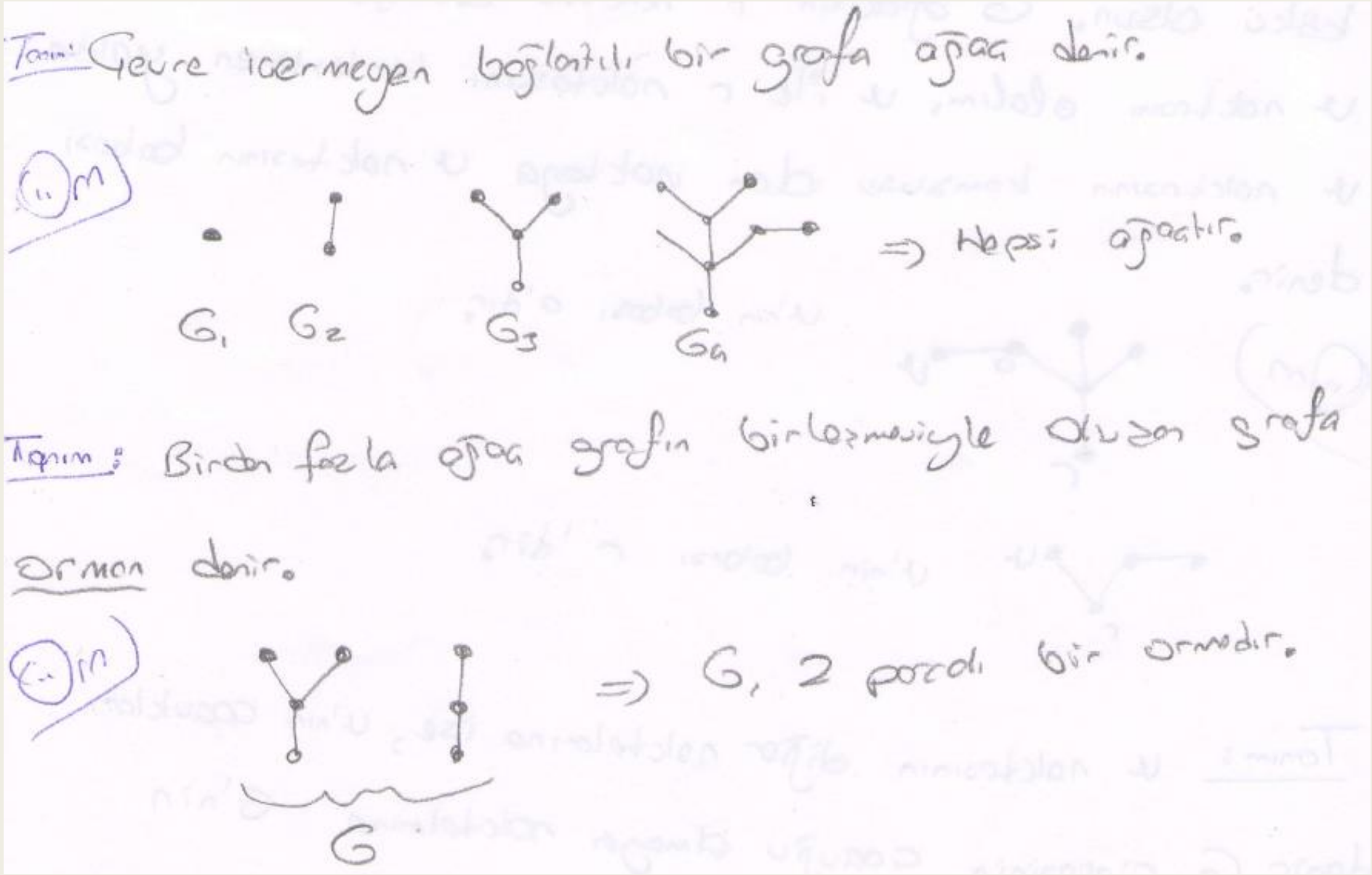
Sonuç olarak: içten ayrık yolların sayısı 2'dir.



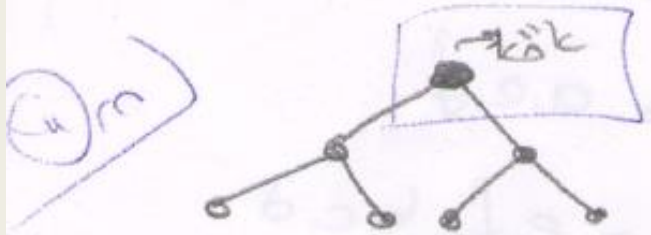

Böylece $k(G) = 2$ 'dir.

~~*~~ içten ayrık yollarda ortak ağırlık yoktur !!!

# Ağaç Graflar ve Ağaç Grafların Bilgisayarlarda Saklanması

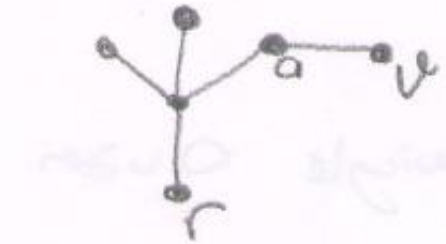


**Tanım:** Çoğu zaman tepelerinden biri özel olan ağaçlar ile çalışılır. Özel olarak seçilen bu tepeye kök denir. Bu grafta köklü ağaç denir.




Köklü ağaç

**Tanım:** G köklü bir ağaç ve r noktası da G'nin kökü olsun. G ağacının r noktasından farklı olan bir v noktasını alalım. v ile r noktasını birleştiren yolda v noktasının komşusu olan noktaya v noktasının babası denir.




v'nin babası a'dır.



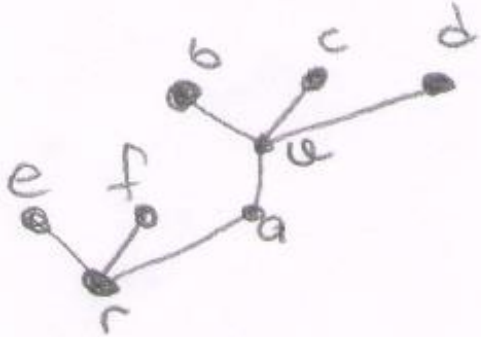

v'nin babası r'dir.

**Tanım:** $u$ noktasının diğer noktalarına ise, $u$'nun çocukları denir. $G$ çizgesinin çocuğu olmayan noktalarına $G$'nin yaprakları denir.

1 dereceli tepelere $G$'nin yaprakları denir.

**Uyarı:** $r$ bir kök olmak üzere $r$'nin babası yoktur.

(a,n)




G grafı

$u$'nun babası = a

çocukları = b, c, d

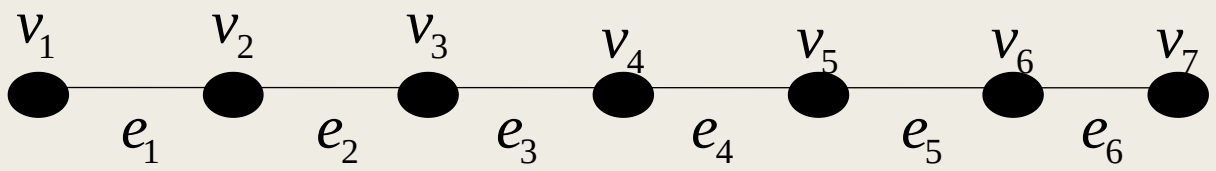
$r$'nin çocukları = a, e, f

$G$'nin yaprakları = e, f, b, c, d

**Teorem:** *G*, *p* tepeli *q* ayrıtlı bir ağaç graf olsun. Bu durumda $p=q+1$ dir.

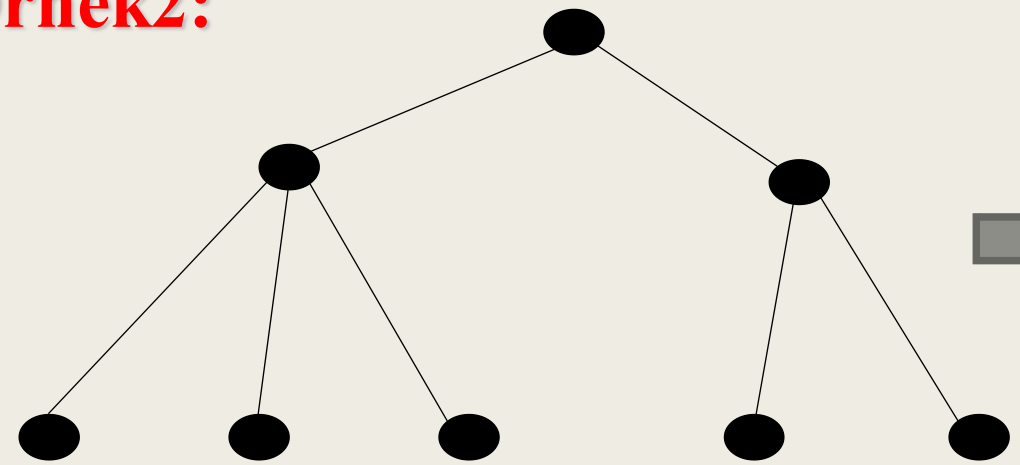
**Kanıt:** Tümevarım yardımıyla kanıt yapılır.

**Örnek1:**




$P_7$ Grafı ⇒ 7 tepeli, 6 ayrıtlıdır. Böylece, 7=6+1 elde edilir.

**Örnek2:**



T Ağacı

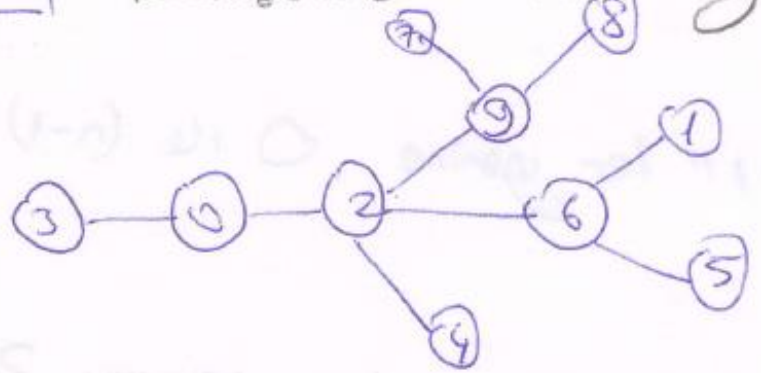
⇒ T ağacı; 8 tepeli, 7 ayrıtlıdır. Böylece, 8=7+1 elde edilir.

= Ağaçları Bilgisayar Belleğinde Nasıl saklarız? =

4 farklı yöntem ile ağaçları bilgisayar belleğinde saklaya-

bilirsiniz.

1. yöntem: Komşuluk matrisi yardımıyla.



G

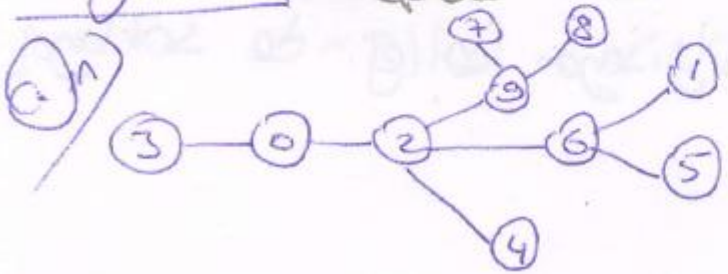
G ağacını aşağıdaki matris yardımıyla bellekte saklayabilirsiniz.

G ağacını adjacency matrix yardımıyla bellekte saklayabiliriz.

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 2 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 |
| 3 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 4 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 6 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 7 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 8 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| 9 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

(*) Bu gösterimle; n tepeli bir ağacı saklamak için $n^2$ bit gerekli dir. Fakat matris simetrik olduğundan ve köşeden elemanlar sıfır olduğundan $\frac{n^2-n}{2}$ bit yeterlidir.

2. yöntem / Tepeleri sıklamayıp, sadece kenarları sıklarız.




Böylece, bu yöntemde sadece 2 satır kullandık ve (n-1) sütun kullanıldı.

Ancak bu kez, "0" ile "1" ler yerine 0 ile (n-1) arasındaki tamsayıları kullandık.

(*) n pozitif bir tamsayı olmak üzere bu sayıyı 2'lik sistemde yazmak için $\lceil \log_2 n \rceil + 1$ bit gerekir.

Bu durumda; n tepeli bir ağacı bellekte sıklamak için;

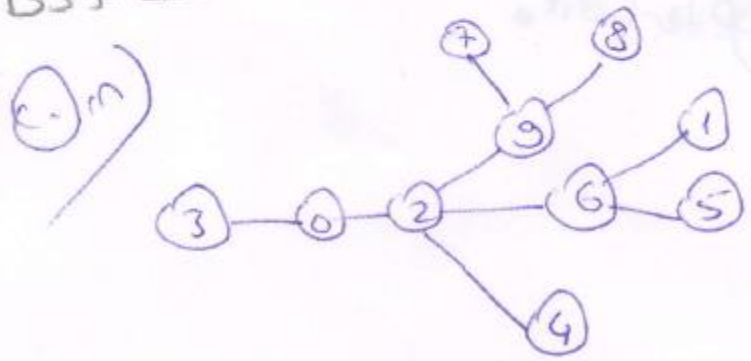
$2.(n-1).(\lfloor \log_2 n \rfloor + 1) \leq \frac{n^2-n}{2}$

Böylece, 2. yöntem için daha az bit gereklidir.

3. yöntem (Father Code)

Bu yöntemde 0 nolu tepeyi kök kabul ediriz.

2. yönteme benzer olarak yine kenarlar sıkılır. Ancak kural üst sıraya çocuklar, alt sıraya baba'lar yazılır.




| çocuklar | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| babalar | 6 | 0 | 0 | 2 | 6 | 2 | 9 | 9 | 2 |

Bu yöntemde,

→ O(kök) üst satırda bulunmaz.

→ O harfi tüm noktaları üst satırda bulunmak zorundadır.

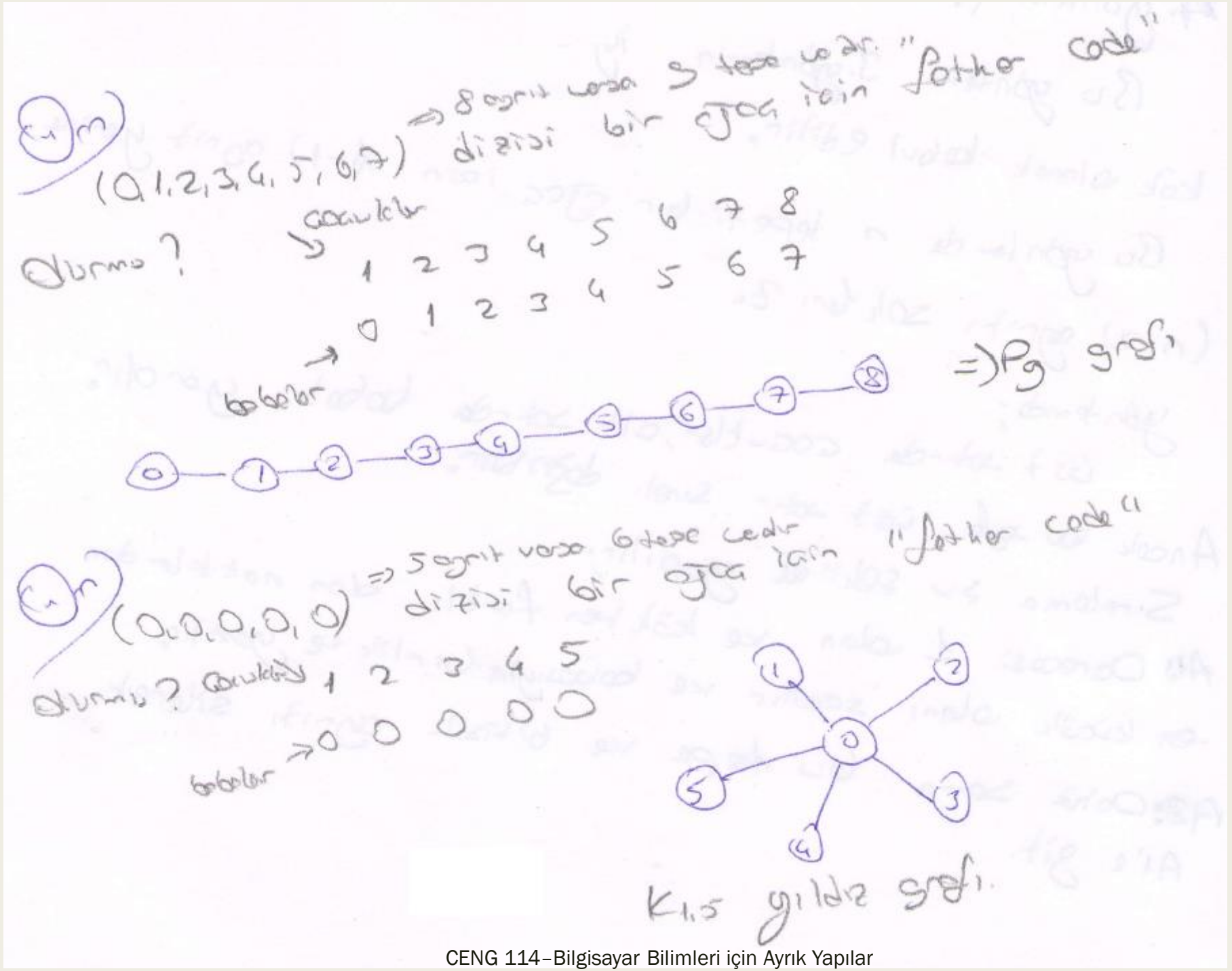
→ Üst satırda hiç bir rakam 2 kez kullanılmaz.

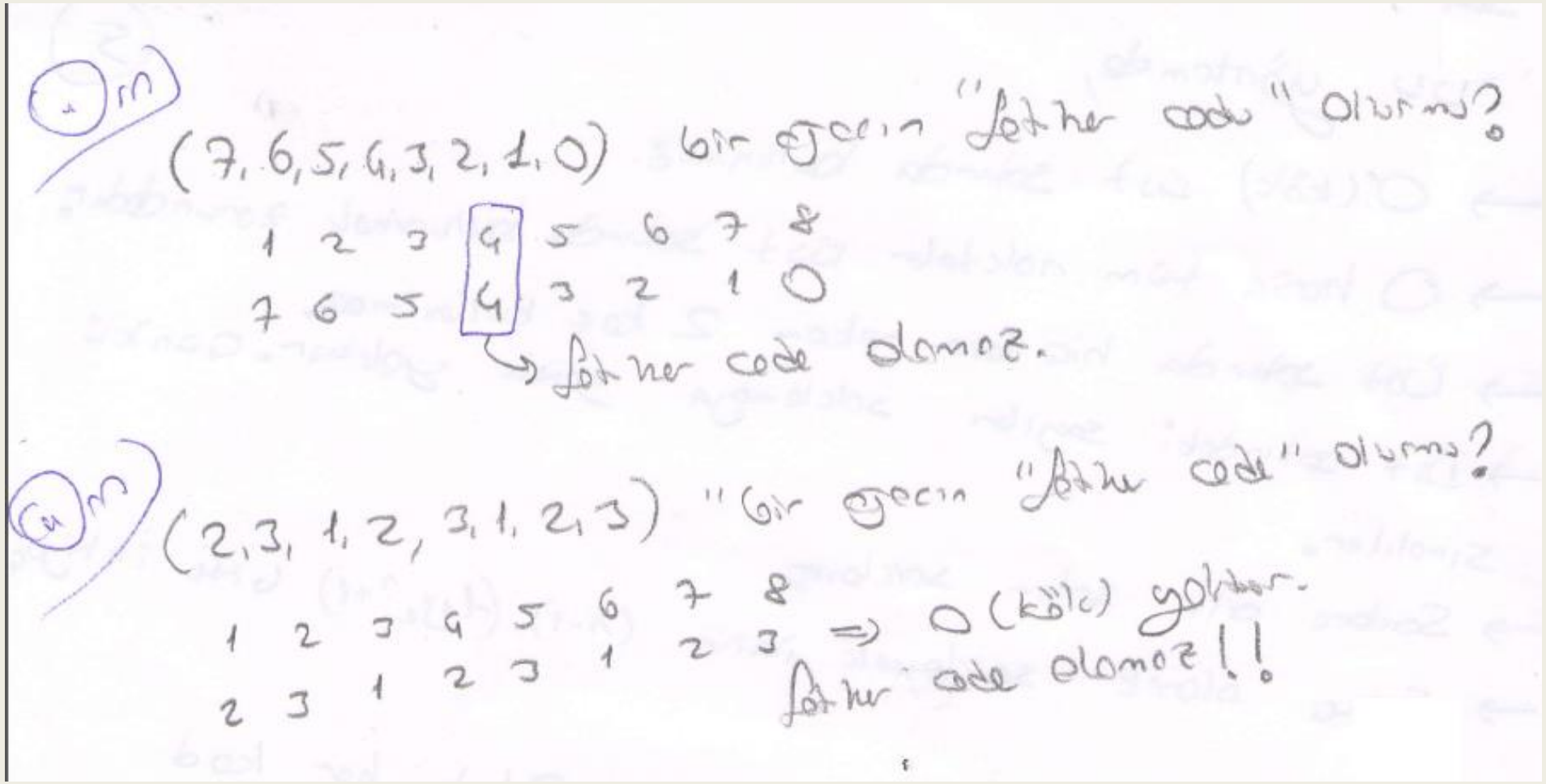
→ Üst satırdaki sayıları soldaya gerek yoktur. Çünkü sıralılar.

→ Sadece alt satır soldanır.

→ Sonuç olarak soldamak için $(n-1).(\lfloor \log_2 n \rfloor + 1)$ bite ihtiyaç vardır.

(*) Her grafa için bir kod yazılabilir. Fakat her kod bir grafa bakıştırmayabilir!!


(a) m
(0,1,2,3,4,5,6,7) dizisi bir ağaç için "Father code"
8 eşit sayıda 9 tepe vardır.
Ölürme?
çocuklar
1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7
ebeler
=) Pg grafı
(b) n
(0,0,0,0,0) dizisi bir ağaç için "Father code"
5 eşit sayıda 6 tepe vardır.
Ölürme? çocuklar
1 2 3 4 5
0 0 0 0 0
ebeler
K1,5 yıldız grafı.


(7, 6, 5, 4, 3, 2, 1, 0) bir ağacın "left heavy" olur mu?
1 2 3 4 5 6 7 8
7 6 5 4 3 2 1 0
left heavy olamaz.
(2, 3, 1, 2, 3, 1, 2, 3) bir ağacın "left heavy" olur mu?
1 2 3 4 5 6 7 8
2 3 1 2 3 1 2 3
⇒ O(kök) yoktur.
left heavy olamaz!!

4. yöntem (Prüfer Kod)

Bu yöntem 3. yöntemin iyileştirilmesidir. O yine kök olarak kabul edilir.

Bu yöntemde n tepeli bir ağaç için (n-1) ağrıt yerine (n-2) ağrıtı saklarız.

yöntemde;

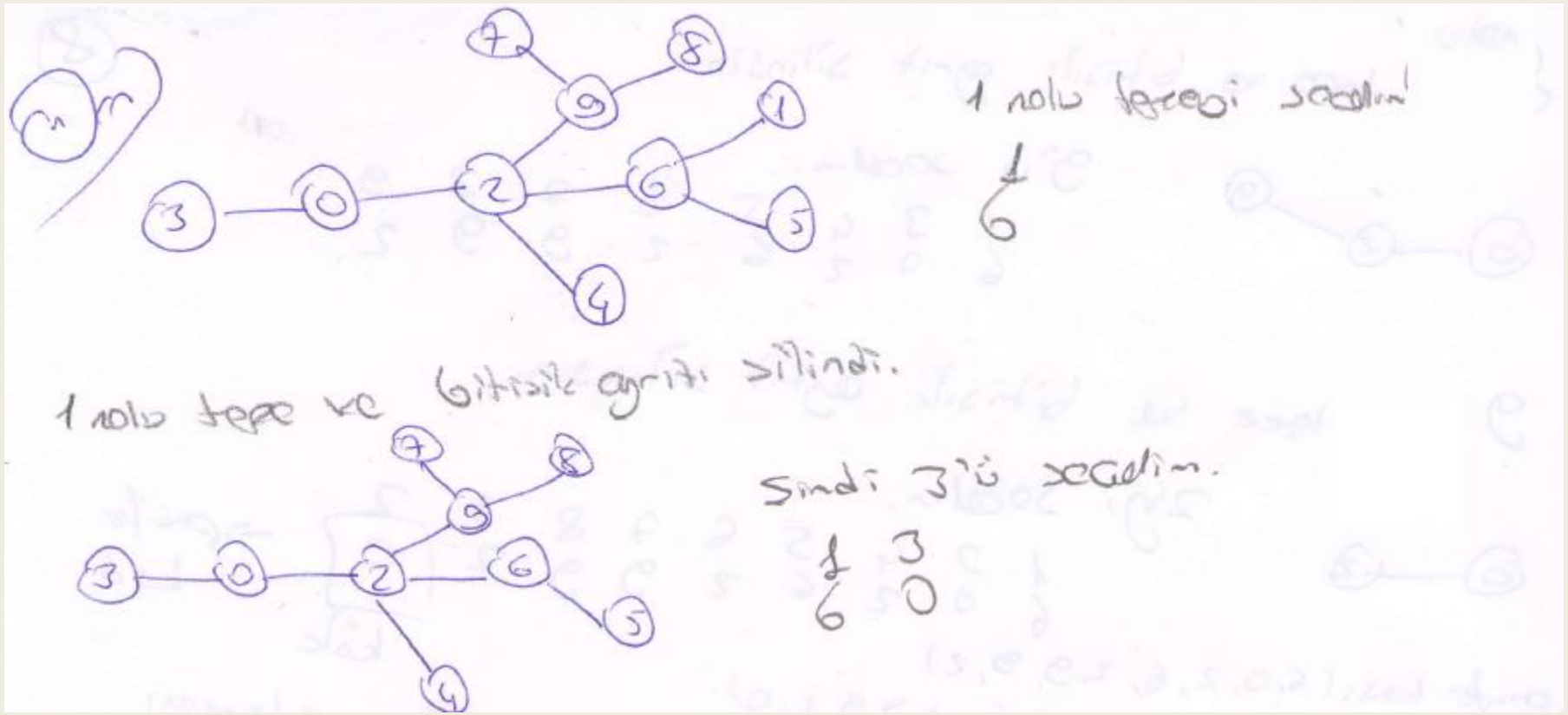
üst satırda çocuklar, alt satırda babalar yazılır.

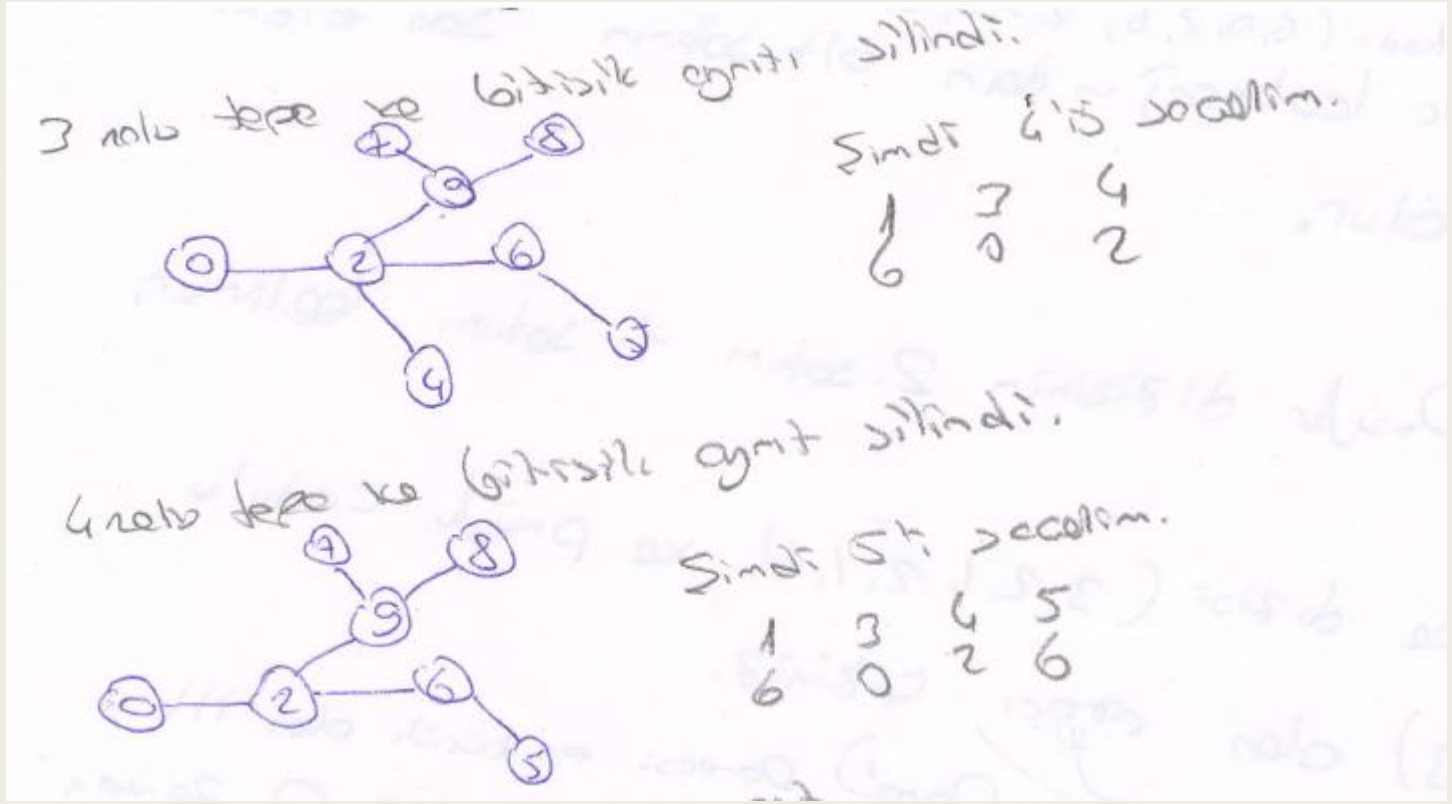
Aradaki bu ilişki üst satır sıralı değildir.

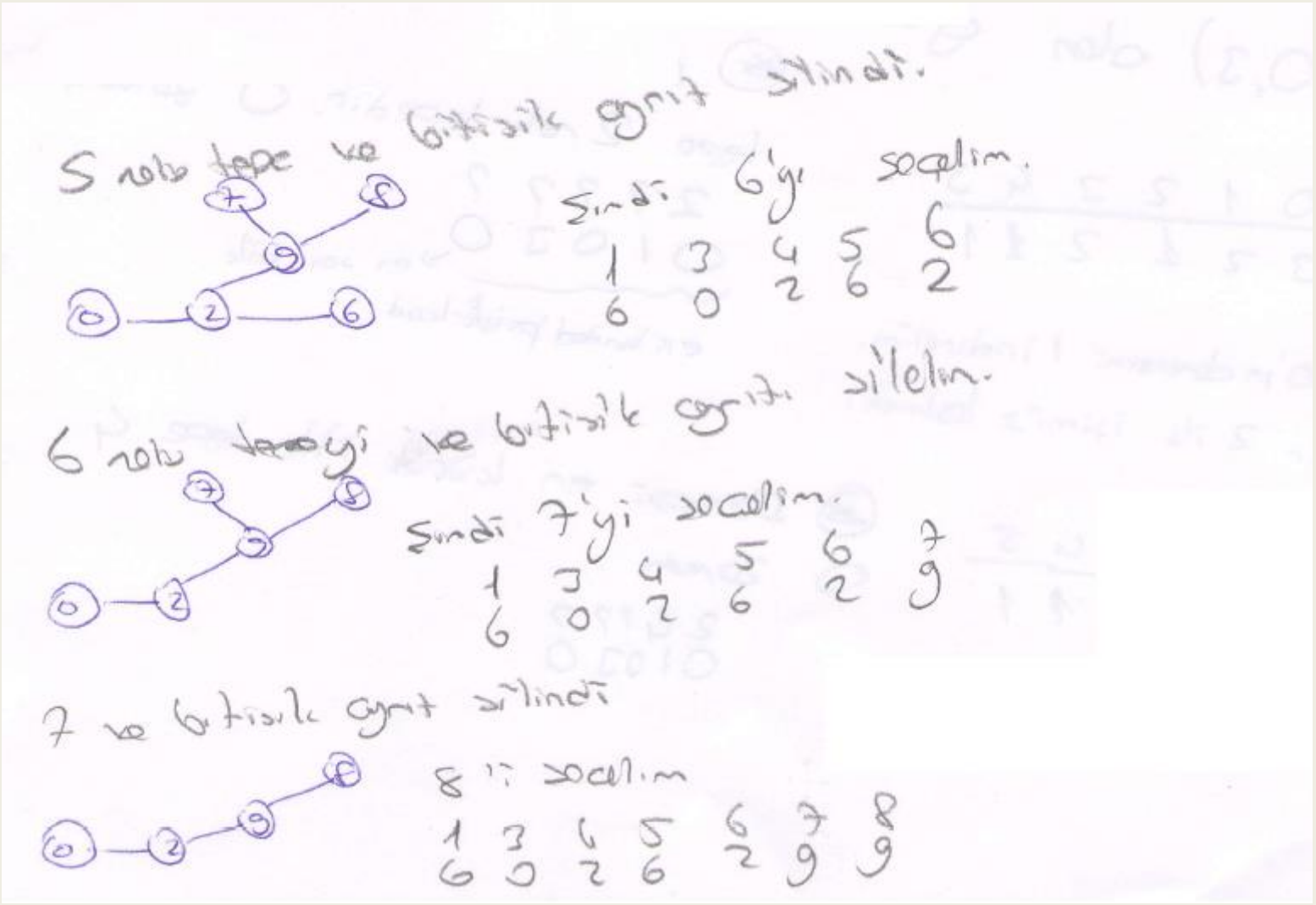
Sıralama şu şekilde yapılır;

A1: Derecesi 1 olan ve kökten farklı olan noktalardan en küçük olanı seçilir ve babasıyla birlikte yazılır.

A2: Daha sonra bu tepe ve bitişik ağrıtı silerek A1'e git.


1 nolu tepeyi seçelim
1 nolu tepe ve bitişik ağrıtı silindi.
Şimdi 3'ü seçelim.


3 nolu tepe ve bitişik eşrit silindi.
Şimdi 4'ü yazalım.
1 3 4
6 0 2
4 nolu tepe ve bitişik eşrit silindi.
Şimdi 5'i yazalım.
1 3 4 5
6 0 2 6


5 nolu tepe ve bitişik ayrıt silindi.
Şimdi 6'yı sorgulayalım.
1 3 4 5 6
6 0 2 6 2
6 nolu tepeyi ve bitişik ayrıtı silelim.
Şimdi 7'yi sorgulayalım.
1 3 4 5 6 7
6 0 2 6 2 9
7 ve bitişik ayrıt silindi
8'i sorgulayalım
1 3 4 5 6 7 8
6 0 2 6 2 9 9

Final tek ve bitişik eşit silinsin.



9'u seçelim.

| 1 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|
| 6 | 0 | 2 | 6 | 2 | 9 | 9 | 2 |

9 nolu tepe ve bitişik eşit silinsin.



2'yi seçelim.

| 1 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 2 |
|---|---|---|---|---|---|---|---|---|
| 6 | 0 | 2 | 6 | 2 | 9 | 9 | 2 | 0 |

→ prüfer kod

kök

prüfer kod: (6, 0, 2, 6, 2, 9, 9, 2)

Extended prüfer kod = (6, 0, 2, 6, 2, 9, 9, 2, 0)

En son kök kalacağından alt satırın son elemanı

O(sıfır) olur.

**Teorem:** Prüfer dizisinin 2. satırı 1. satırı bölmez.

(n)

Derece dizisi (3,2,1,2,1,1) ve prüfer kodu (0,1,0,3) olan ağacı çiziniz.

| Tepe | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| Derece | 3 | 2 | 1 | 2 | 1 | 1 |

Şimdi 2'ye 0'ın derecesini 1 indirelim.

Sıfır old. dan 2 ile bizim ile bölmedi.

| Tepe | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| Derece | 2 | 2 | 0 | 2 | 1 | 1 |

(*) Derecesi en küçük olan ilk tepe 2 nolu tepedir. 0 sonra

2 ? ? ? ?
0 1 0 3 0 → en son kalan

extended prüfer kod.

(*) Derecesi en küçük ilk tepe 4

0 sonra

2 4 ? ? ?
0 1 0 3 0

4'ün ve 1'in derecesini 1 azaltalım.

En küçük dereceli ilk tepe = 1

| Tepe | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| Derece | 2 | 1 | 0 | 2 | 0 | 1 |

2 4 1 ? ?
0 1 0 3 0

Şimdi 0'ın ve 1'in derecesini 1 azaltalım.

0 komşu (kök) en küçük dereceli: tepe=5

| Tepe | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| Derece | 1 | 0 | 0 | 2 | 0 | 1 |

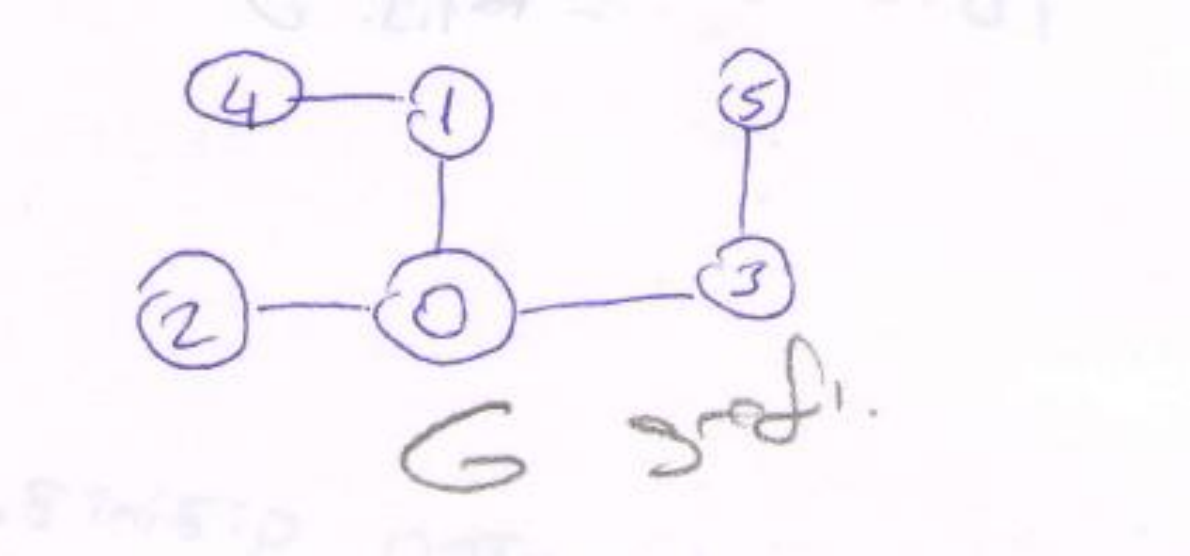
2 4 1 5 ?
0 1 0 3 0

Şimdi 5 ve 3'ün derecesi 1 azalsın.

| Tepe | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| Derece | 1 | 0 | 0 | 1 | 0 | 0 |

0 komşu derecesi en küçük
tepe = 3

2 4 1 5 3 ✓
0 1 0 3 0


4
1
5
2
0
3
G grafı.

Tanım: (Düzlemsel Kod)

Tepe sayısı $n$ olan bir çizge alalım. Bu çizgenin kökü $r$ olsun ve $r$'den başlayıp çizgenin tüm ağıtlarından 2 kez geçerek bir tur yapalım. Bu turda ulaştığımız tepe bir önceki tepenin çocuğu ise bunu "1" ile, öncekini ise "0" ile göstererek elde edilen koda düzlemsel kod denir.

Örnek




Tuvmoz

refgfecbgabcdeeb

1 1 1 0 0 1 1 1 0 0 1 0 0 0

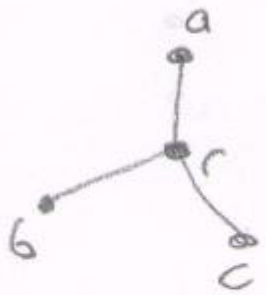
(Ön)

1111100000 ödevsel kodum kesinlik gelen ağaç.




= P6 yol grafı

(a) 101010 dizisel kodun basitlik şeklen ağaç.




carbrcr
vvvvvv
101010 = $K_{1,3}$ yıldız graf.

(b) 11000111100 dizisel kodlu ağacı çiziniz.

En başta 2 tane 1, fakat sonra arka arkaya 3 sıfır olduğundan böyle bir ağaç çizilemez.

# Kaynaklar


- *1- Discrete Mathematics and Its Applications*, Kennet H. Rosen

(Ayrık Matematik ve Uygulamaları, Kennet H. Rosen (Türkçe çeviri), Palme yayıncılık)

- *2- Discrete Mathematics: Elementary and Beyond*, L. Lovász, J. Pelikán, K. Vesztergombi, 2003.
- *3- Introduction to Algorithms*, T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein, 2009.
- *4- Introduction To Design And Analysis Of Algorithms*, A. Levitin, 2008.
- *5-Algoritmalar (Teoriden Uygulamalara)*, Vasif V. Nabiyev, Seçkin Yayıncılık